Companion to Bird & de Moor, Algebra of Programming

§7. Optimisation Problems

§7.1. Minimum and maximum

(7.1a)

`est(R) ≜ ∋ ∩ (∈ \ R°)` — `x` is the `est(R)` of `xs` when `x∈xs` and `x R y` for every `y∈xs`; B&dM call it `min R`.

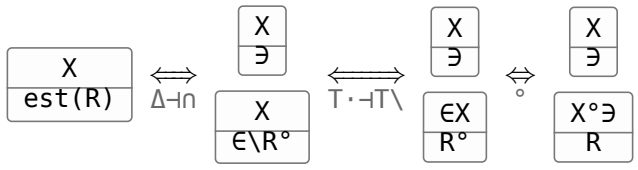
`est(≤) xs (est(≤)) x⇔x∈xs∧(∀y∈xs. x≤y).`

(7.1b)

	the law	what it says
1	`X⊑est(R)⇔X⊑∋` and `X°∋⊑R`	in the set, and below every element of it
2	$\frac{\mathbb{1}}{\ni}$ `(∈\R)=R`	bounding a singleton is bounding its element
3	$\frac{S}{\ni}$ `(∈\R)=S°\R`	bound `S`'s image without building the set
4	`union ≜` $\frac{\ni\ \ni}{\ni}$	flattens a set of sets
5	`union (∈\R)=∈\(∈\R)`	bound a union by bounding each member set
6	$\frac{\mathbb{1}}{\ni}$ `est(R) = 𝟙 ∩ R°` (7.4)	a singleton's minimum is its element, where `R` is reflexive $\frac{S}{\ni}$ `est(R)` at `S:=𝟙`
7	$\frac{S}{\ni}$ `est(R) = S ∩ (S° \ R°)`	an `S`-value that points to every `S`-value
8	$\frac{S}{\ni}$ `est(R ∩ (S° S)) =` $\frac{S}{\ni}$ `est(R)`	only `R` between values `S` gives one argument counts — context
9	`E(S) est(R)=(∋S)∩((∋S)°\R°)` (7.7)	the same for the image of a set $\frac{S}{\ni}$ `est(R)` at `S:=∋S`
10	`Map(f) ⇒ P(f) est(R) = est(f R f°) f`	shunt a function through a minimum
11	`P(S) est(R)=(∋S)∩(∈\(SR°))` (7.9) `R` reflexive	fusion with the power relator `⊒` is the only proof here that tabulates
12	`P(S) est(R)⊑(∋S)∩(∈\(SR°))`	the half of the row above that costs nothing
13	`P(est(R)) est(R)⊑union est(R)` `R` transitive	a minimum in each set, then a minimum of those
14	`R² ⊑ R ⇒ P(est(R)) est(R) = P(dom(est(R))) union est(R)` (7.12)	the same as an equality, once empty sets are dropped

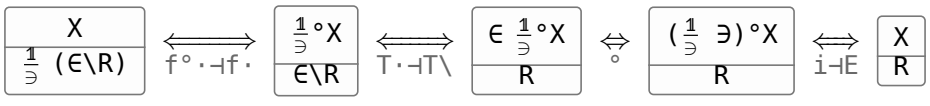
§7.1.1. `X ⊑ est(R) ⇔ X ⊑ ∋ ∧ ∈ X ⊑ R°` — X refines est(R) when it picks a member R-related to every member

(7.1.1a)



§7.1.2. $\frac{\mathbb{1}}{\ni}$ `(∈ \ R) = R` — all of {a} relates by R to c exactly when a does

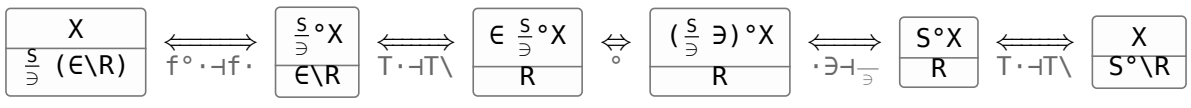
(7.1)



§7.1.3. $\frac{S}{\ni}$ `(∈ \ R) = S° \ R` — gathering the S-values into one set first changes nothing

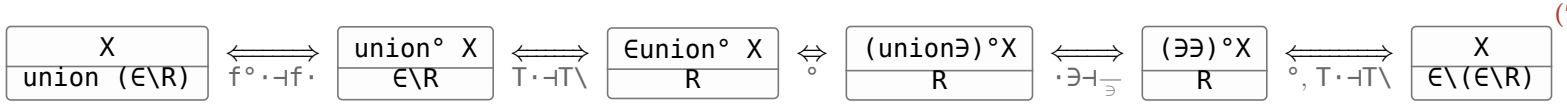
$\frac{S}{\ni}$ gathers the `S`-image of a point into one set and `∈\R` asks that every member of that set be `R`-related to the target, so the set cancels and `S°\R` asks it of the `S`-image directly.

(7.2)



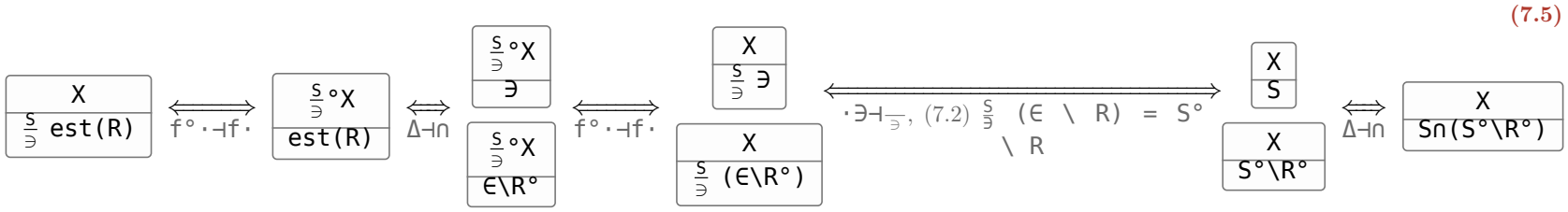
§7.1.4. `union (∈ \ R) = ∈ \ (∈ \ R)` — every member of the union relates by R iff every set's members do

(7.3)



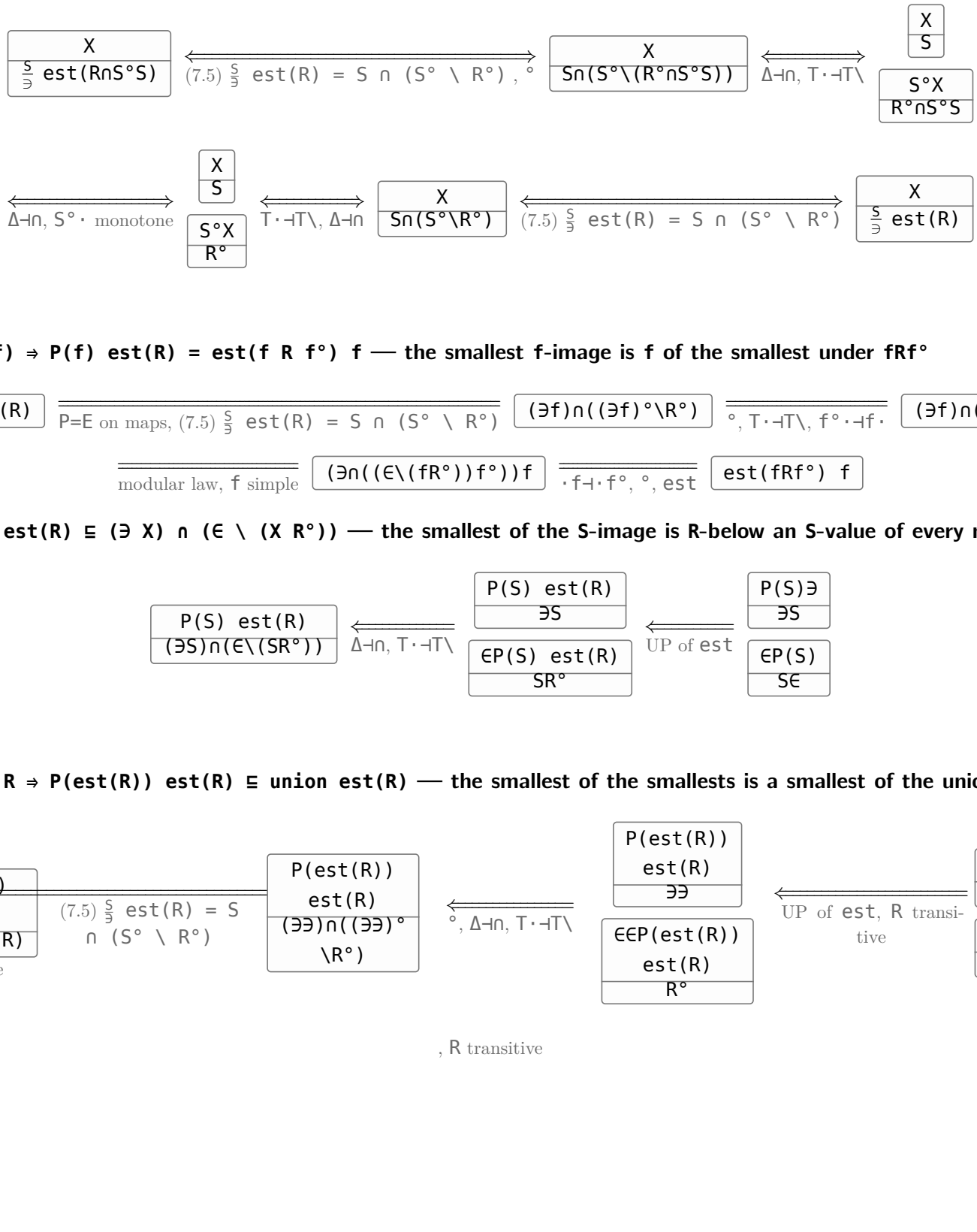
§7.1.5. $\frac{S}{\ni}$ `est(R) = S ∩ (S° \ R°)` — the smallest S-value is an S-value R-related to every S-value

(7.5)



§7.1.6. $\frac{S}{\ni}$ `est(R ∩ (S° S))` = $\frac{S}{\ni}$ `est(R)` — only R between S-values of one argument matters for the smallest

(7.6)



§7.1.7. `Map(f) ⇒ P(f) est(R) = est(f R f°) f` — the smallest f-image is f of the smallest under fRf°

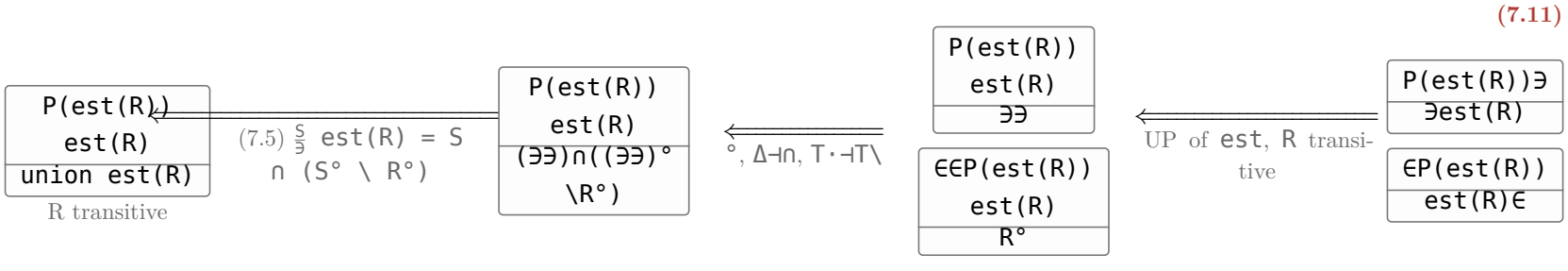
(7.8)

§7.1.8. `P(X) est(R) ⊑ (∋ X) ∩ (∈ \ (X R°))` — the smallest of the S-image is R-below an S-value of every member

(7.10)

§7.1.9. `R² ⊑ R ⇒ P(est(R)) est(R) ⊑ union est(R)` — the smallest of the smallests is a smallest of the union

(7.11)



§7.2. Monotonic algebras

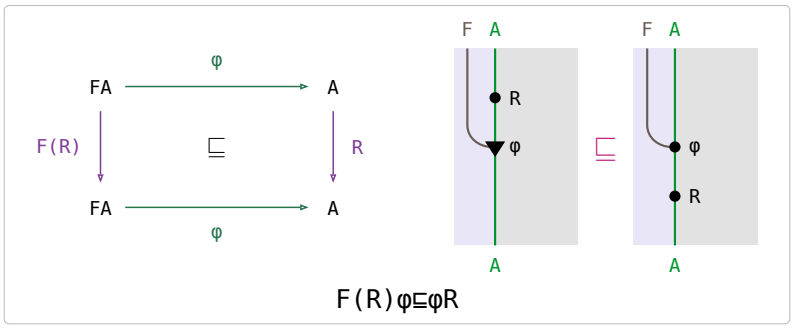
(7.2a)

φ Pres R ≜ F(R) φ ⊑ φ R — **φ preserves R**; B&dM call **φ monotonic on R**.

- if xs F(R) ys and φ merges ys to y, then φ merges xs to some x with x R y

(≤x≤)+⊑+≤ — addition, Alg(Δ,ℕ) , preserves ≤: c=a+b∧a≤a'∧b≤b'⇒c≤a'+b'.

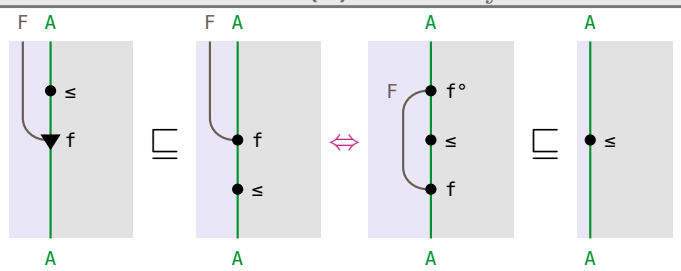
(7.2b)



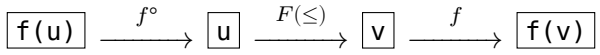
§7.2.1. f monotonic on ≤, read at points

F(≤) f ⊑ f ≤ ⇔ f° F(≤) f ⊑ ≤ (7.2.1a)

F(≤) then f stays below f then ≤ exactly when f°, F(≤), f in a row stays below ≤.



1	F(≤) f ⊑ f ≤	Map(f) ⇒ f° R ⊑ ≤ ⇔ R ⊑ f ≤
2	⇔ f° F(≤) f ⊑ ≤	



- F(≤): from u to any v above it; u, v are in FA, compared component by component. This step is the premise.
- ⊑≤: the two ends satisfy f(u)≤f(v), the conclusion. So u F(≤) v ⇒ f(u)≤f(v): f preserves ≤.

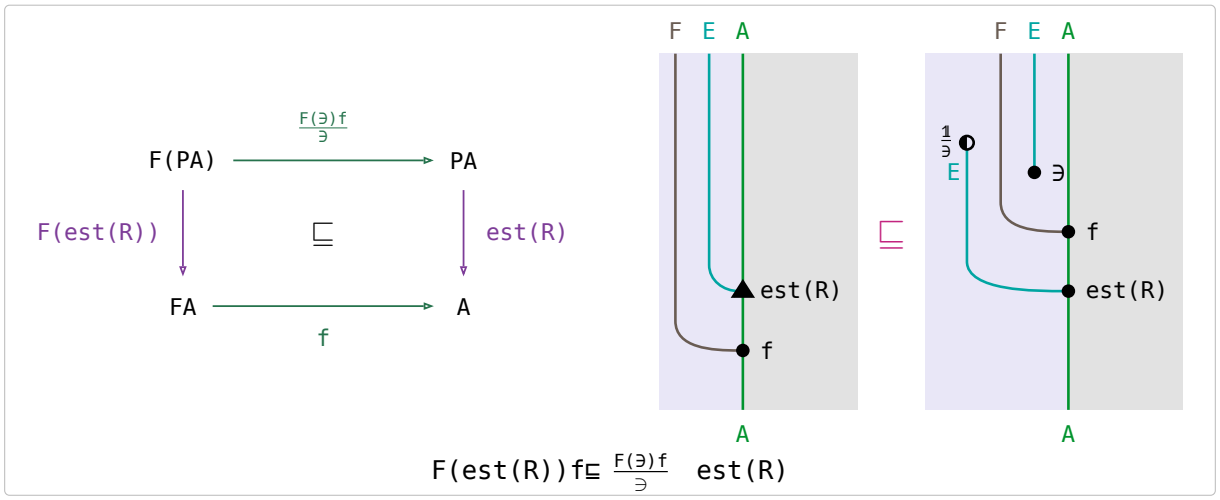
§7.2.2. Function f is monotonic on R iff it distributes over R

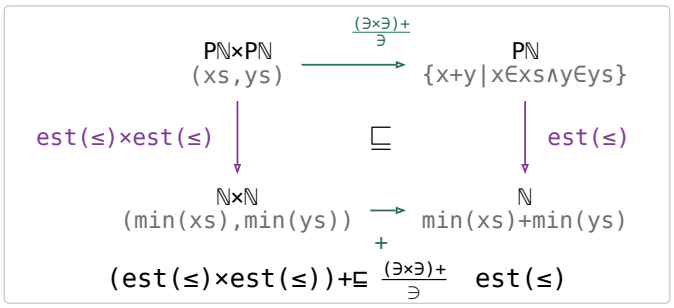
(7.2.2a)

f : FA→A **distributes over** R when F(est(R))f⊑ $\frac{F(∋)f}{∋}$ est(R).

(est(≤)×est(≤))+⊑ $\frac{(∋×∋)+}{∋}$ est(≤) — addition distributes over ≤: min(xs)+min(ys) is a least element of {x+y|x∈xs∧y∈ys}.

(7.2.2b)

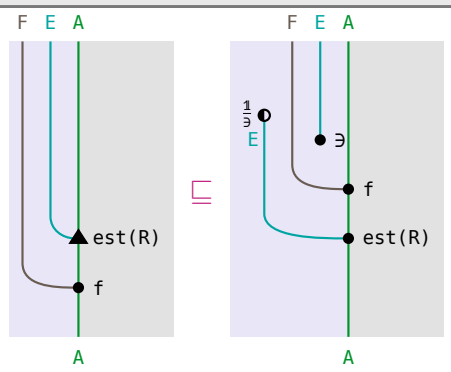




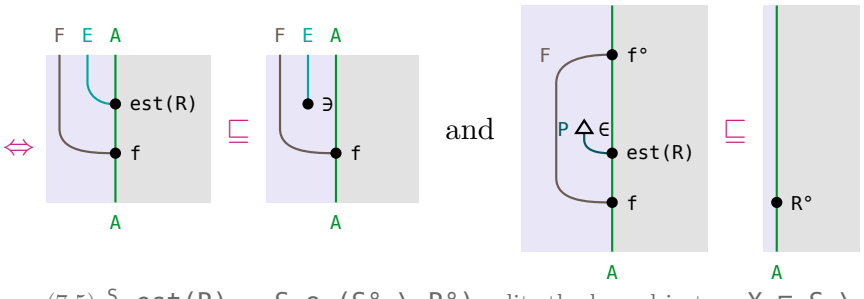
Map(f); 𝟙 ⊑ R — Theorem 7.1

⇒ f° F(R) f ⊑ R ⇔ F(est(R)) f ⊑ F(∋) f / ∋ est(R)

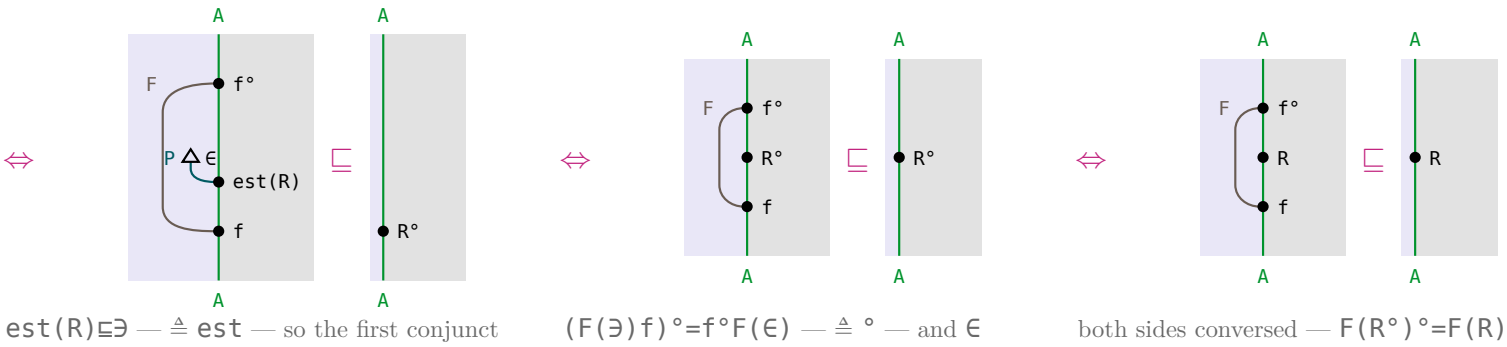
f carries **R**-related parts to **R**-related wholes (**f°F(R)f⊑R**) if and only if applying **f** to the **R**-best parts (**F(est(R))f**) is included in applying **f** to all parts, then taking an **R**-best (**F(∋)f** then **est(R)**)



f distributes over R — ≜ distributes — the fraction bent as R/∋ ∋ = R



(7.5) S/∋ est(R) = S ∩ (S° \ R°) splits the bound in two, X ⊑ S \ R ⇔ S X ⊑ R moving (F(∋)f)° across



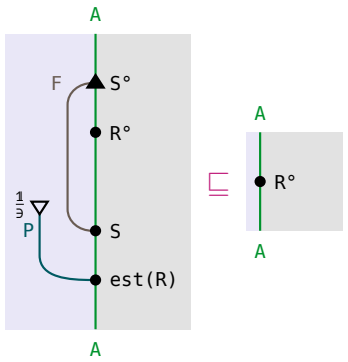
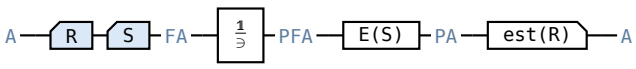
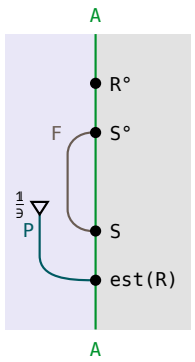
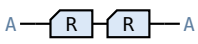
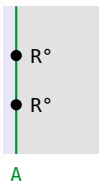
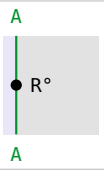
est(R)⊑∋ — ≜ est — so the first conjunct drops

(F(∋)f)°=f°F(∈) — ≜ ° — and ∈ est(R)=R° — ≜ est, R reflexive

both sides conversed — F(R°)°=F(R), F(R°) = F(R)°

§7.2.3. R² ⊑ R; S Pres R ⇒ ⦇S/∋ est(R)⦈ ⊑ ⦇S⦈/∋ est(R) — taking the best at each step gives a best overall result

Theorem 7.2

circuit	Hinze–Marsden
⦇S/∋ est(R)⦈⊑⦇S⦈/∋ est(R) the conclusion: one minimum kept at each step is below every result collected and one minimum taken at the end	
⦇S/∋ est(R)⦈ ⊑ ⦇S⦈ and ⇔ ⦇S⦈°⦇S/∋ est(R)⦈ ⊑ R° (7.5) S/∋ est(R) = S ∩ (S° \ R°) at ⦇S⦈; ⦇S/∋ est(R)⦈⊑⦇S⦈°\R°⇔⦇S⦈°⦇S/∋ est(R)⦈⊑R° — X ⊑ S \ R ⇔ S X ⊑ R	
⟹S/∋ est(R)⊑S the left conjunct; (7.5) S/∋ est(R) = S ∩ (S° \ R°) , and ⦇−⦈ is monotone	
 ⟹the right conjunct; ⦇S⦈°⦇S/∋ est(R)⦈ is the least X with X=S°F(X)(S/∋ est(R)) hylomorphism theorem — so Knaster–Tarski leaves this one inequation S/∋ = 1/∋ E(S) — R/∋ ∋ = R	
⊑  S°F(R°)⊑R°S° — ≙ monotonic at S, conversed; F(R)°=F(R°) — F(R°) = F(R)°	
⊑  S°(S/∋ est(R))⊑S°(S°\R°)⊑R° — (7.5) S/∋ est(R) = S ∩ (S° \ R°) , (0.9a.4)	
⊑  R transitive	

§7.2.4. `Coreflexive(p) ⇒ takewhile(p) = ⦇[nil,twStep(p)]⦈` — `takewhile(p)` keeps items while `p` holds, as one fold

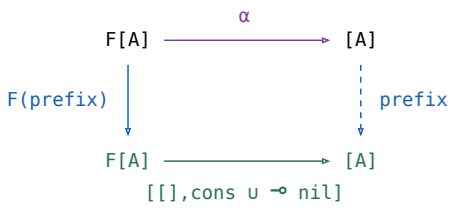
(7.2.4a)

	definition	type	example and in words
1	`FX = L + E × X`	`𝐴→𝐴`	nothing, or a head and a tail
2	`nil ≜ []`	`𝟙→[A]`	the empty list — the first constructor of `[A]`
3	`cons(a,[x₁,…,xₙ]) ≜ [a,x₁,…,xₙ]`	`A×[A]→[A]`	`cons(3,[1,2])=[3,1,2]` — a head onto a tail, the second constructor
4	`α ≜ [nil,cons]`	`Alg(F,[A])`	both constructors as one map
5	`p ⊑ 𝟙`	`A→A`	`p≜even` — `2 p 2`, and `3∉dom(p)` — a coreflexive: `a` passes `p` when `a p a`
6	`R ≜ length ≤ length°`	`[A]→[A]`	`[1] R [1,2]` — `xs R ys⇔length(xs)≤length(ys)`
7	`(a,xs) ↦ []`	`A×[A]→[A]`	`(⊸ nil)(3,[1,2])=nil` — drop the pair, return `nil`
8	`prefix = ⦇[[],cons ∪ ⊸ nil]⦈`	`[A]→[A]`	`[3,1,2] prefix [3,1]` — `xs prefix ys⇔∃zs. xs=ys#zs` — at each `cons`, keep the head or stop
9	`S ≜ [[],((p × 𝟙) cons) ∪ ⊸ nil]`	`Alg(F,[A])`	`(4,[2]) S [4,2]`, and `(3,[2]) S nil` only — `prefix`'s algebra with one extra `p` — keep a head that passes `p`, or stop
10	`(X→R,S) ≜ guard(C,X) [R,S]`	`A→B`	the test `X` picks the branch: `R` where `X` holds, `S` where it does not
11	`nil R=⊤, nil R°=nil`		`nil` is the shortest list — below every list, and above only itself, so it loses every `est(R°)`

`prefix = ⦇[[],cons ∪ ⊸ nil]⦈` (7.2.4b)

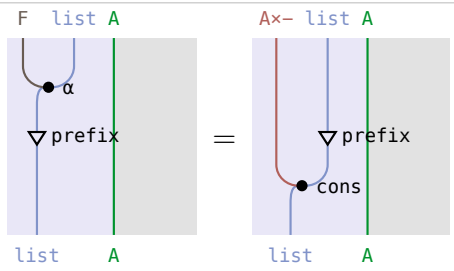
the fold whose algebra, at each `cons`, keeps the head or stops with `nil`: `xs prefix ys⇔∃zs. xs=ys#zs`

the defining square `α prefix=F(prefix)[nil,cons ∪ ⊸ nil]` — build the list and then take a prefix, or take a prefix of the tail and then rebuild with the algebra

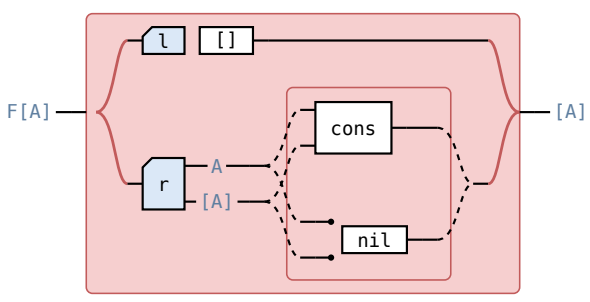


`F=1+A×−`, so `F(prefix)=𝟙+𝟙×prefix`: the head passes, the fold recurses on the tail alone

Hinze–Marsden



the `cons` operand of `cons ∪ ⊸ nil`; `⊸ nil` makes a constant and draws nothing



the algebra `[nil,cons ∪ ⊸ nil] : F([A])→[A]` as a circuit: `nil` on the **1** branch; on a pair `(a,ys′)` two outputs, `nil` and `cons(a,ys′)`

`nil prefix ys⇔ys=nil`
`cons(a,xs′) prefix ys⇔ys=nil ∨ ∃ys′. xs′ prefix ys′ ∧ ys=cons(a,ys′)`

a right fold: `α°` peels, `F(prefix)` recurses, the algebra runs on the way back; `init*` (`prefix = ⦇[[],cons ∪ ⊸ nil]⦈` , `prefix = cat° π₁`) is the tail-recursive form

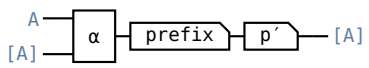
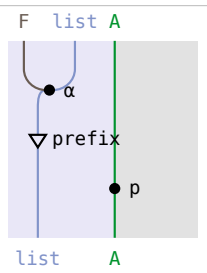
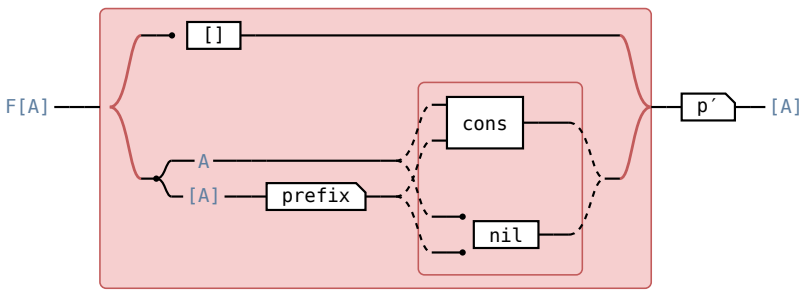
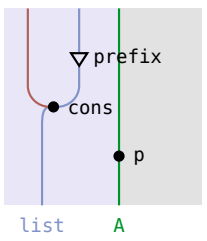
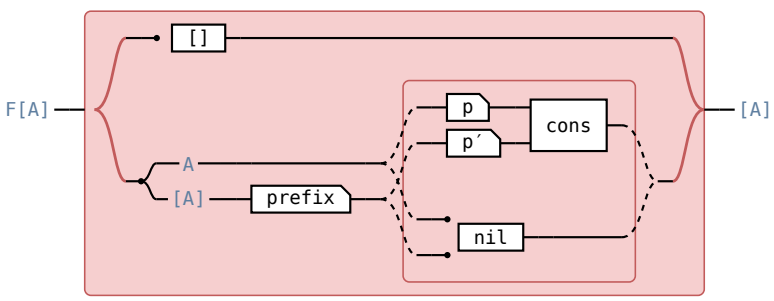
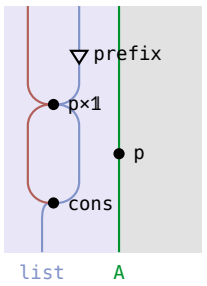
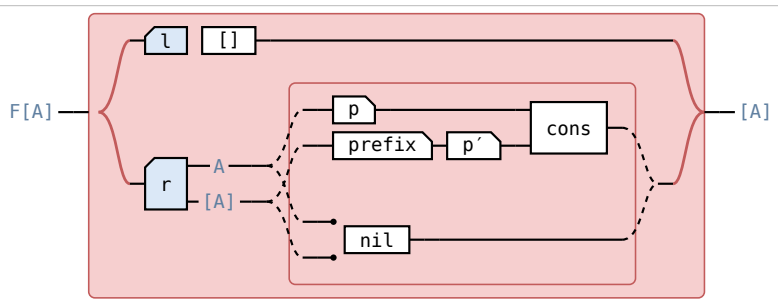
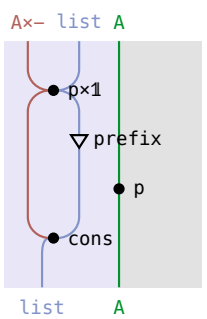
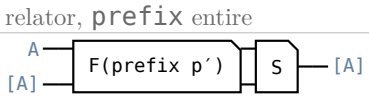
lax natural only, `R′ prefix⊑prefix R′`: `R′` first needs an `R`-image of every element, `prefix` first may have dropped the ones without

Coreflexive(p) (7.2.4c)

⇒ α prefix p′ = F(prefix p′) S

building the list and then keeping a **p**-passing prefix of it is keeping one of the tail first, and then building with **S**

this same diagram is **subseq**'s: algebra **[nil,cons ∪ π₂]**, type **[A]→[A]**

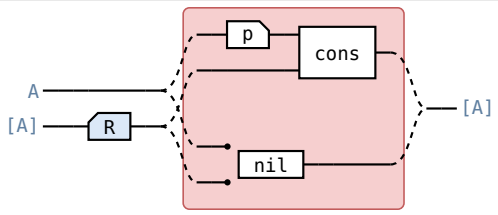
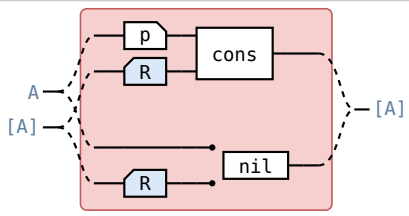
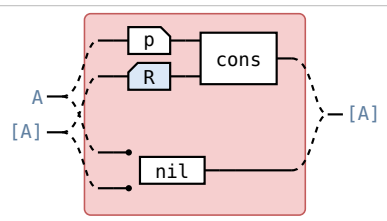
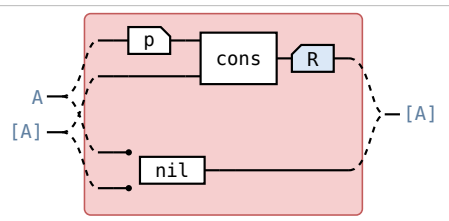
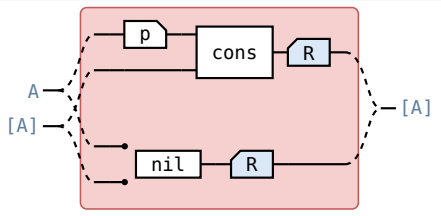
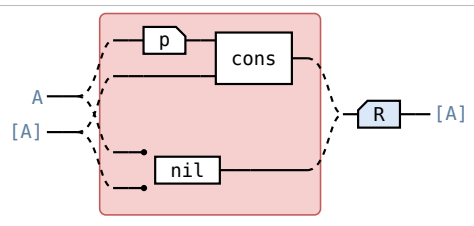
circuit — the fork is `F([A])=1+A×[A]`: `nil` above, the pair below	**Hinze–Marsden**
 `α prefix p′`	 the `cons` branch alone, without `1+` or `⊸ nil`
=  `F(prefix) [[],cons ∪ ⊸ nil] p′` defining equation	 the `cons` operand of `cons ∪ ⊸ nil`
=  `F(prefix) [[],((p × p′) cons) ∪ ⊸ nil]` `p′` through `cons`	 the `(p×p′) cons` operand of `(p×p′) cons ∪ ⊸ nil`
=  `[[],((p × (prefix p′)) cons) ∪ ⊸ nil]` relator, `prefix` entire	
=  `F(prefix p′) S` `prefix p′` entire	

≜ `reduce` reads that off as `prefix p′=(|S|)`. fusion cannot: `p′` is not entire, `(1×p′)⊸ nil⊑⊸ nil`, and no algebra meets the side condition. ,

Coreflexive(p) (7.2.4d)

⇒ `(𝟙 × R°) (((p × 𝟙) cons) ∪ ⊸ nil) ⊑ (((p × 𝟙) cons) ∪ ⊸ nil) R°`

shortening the tail and then taking the step lands inside taking the step and then shortening the result

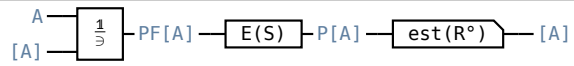
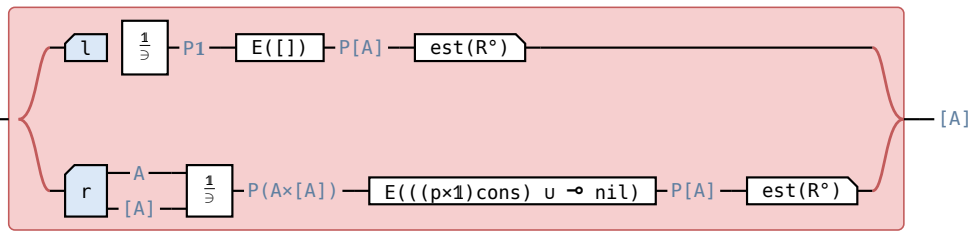
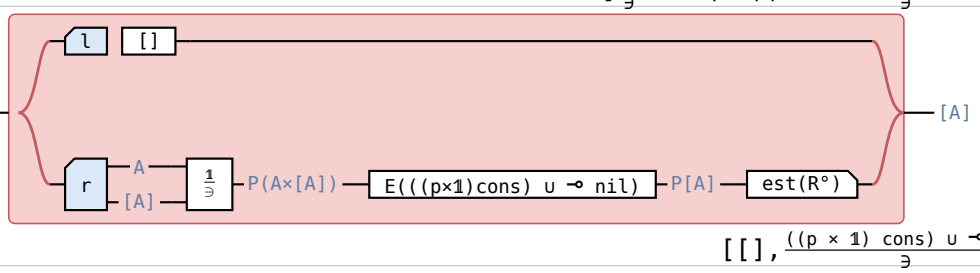
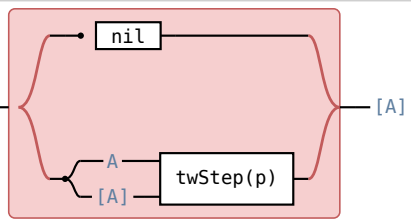
circuit — the `cons` branch of `F(R°)S⊑SR°`	**reason**
 `(𝟙 × R°) (((p × 𝟙) cons) ∪ ⊸ nil)`	
=  `((p × R°) cons) ∪ ((𝟙 × R°) ⊸ nil)`	each operand is reached on its own `(S ∪ T) R = (S R) ∪ (T R)` , `(R ∩ R') / S = (R / S) ∩ (R' / S)` — and `(𝟙×R°)(p×𝟙)` is `p` and `R°` on the pair's two strands at once
⊑  `((p × R°) cons) ∪ ⊸ nil`	`⊸` is the greatest arrow into **1**, so `(𝟙×R°)⊸⊑⊸`
⊑  `((p × 𝟙) cons R°) ∪ ⊸ nil`	`cons length=(𝟙×length)π₂` `succ` with `succ` monotone — a shorter tail makes a shorter list
=  `((p × 𝟙) cons R°) ∪ (⊸ nil R°)`	`nil R°=nil` (7.2.4a.11) — so the constant branch may carry the `R°` the other one already has
=  `(((p × 𝟙) cons) ∪ ⊸ nil) R°`	one `R°` past the join is the two inside it `(S ∪ T) R = (S R) ∪ (T R)` , `(R ∩ R') / S = (R / S) ∩ (R' / S)`

the `nil` branch, which no row above draws, is `nil⊑nil R°`.

Coreflexive(p) (7.2.4e)

⇒ S ∋ est(R°) = [nil,twStep(p)]

the longest of the lists the algebra allows is the **cons** where the head passes **p**, and **nil** where it does not

formula	reason
	
= 	coproduct of maps
= 	singleton, R° reflexive
= 	nil R=⊤

the set is {nil} where p fails on the head and {nil,cons(a,xs)} where it holds, and nil loses the second — ≙ est at a two-element set.

(7.2.4f)

definition	type	example and in words
S ≙ [[],((p × 𝟙) cons) ∪ ⊸ nil]	Alg(F,[A])	(4,[2]) S [4,2], and (3,[2]) S nil only — prefix's algebra with one extra p — stop, or keep a head that passes p

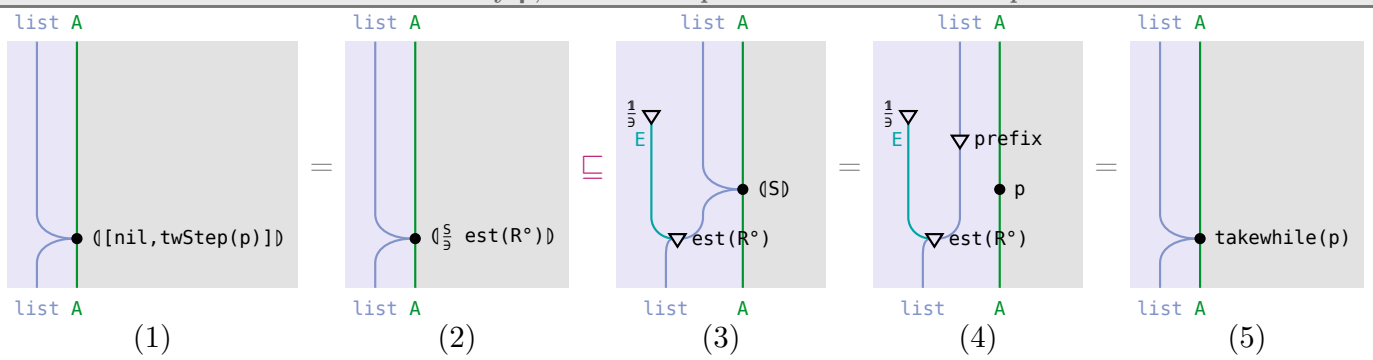
takewhile(p) ≜ $\frac{\text{prefix } p'}{\ni}$ est(R°)

Coreflexive(p)

⇒ takewhile(p) = ⦇[nil,twStep(p)]⦈

Exercise 7.39

takewhile: **takewhile(p)(x)** returns the longest prefix of **x** with the property that all its elements satisfy **p**; the catamorphism is the standard implementation.



1	⦇[nil,twStep(p)]⦈	(7.2.4e) Coreflexive(p) ⇒ $\frac{S}{\ni}$ est(R°) = [nil,twStep(p)]
2	= ⦇$\frac{S}{\ni}$ est(R°)⦈	Coreflexive(p) ⇒ ⦇$\frac{S}{\ni}$ est(R°)⦈ ⊑ $\frac{⦇S⦈}{\ni}$ est(R°)
3	⊑ $\frac{⦇S⦈}{\ni}$ est(R°)	Coreflexive(p) ⇒ prefix p′ = ⦇S⦈
4	= $\frac{\text{prefix } p'}{\ni}$ est(R°)	
5	= takewhile(p)	

takewhile(p)° takewhile(p)⊑prefix° prefix∩R∩R°⊑𝟙

takewhile(p)⊑prefix p′ and (prefix p′)° takewhile(p)⊑R — (7.5) $\frac{S}{\ni}$ est(R) = S ∩ (S° \ R°) at est(R°) — and two prefixes of one list of equal length are equal, so takewhile(p) is simple: **the** longest, not **a** longest.

$\frac{\text{prefix } p'}{\ni}$ est(R°) entire

nil is always a p-prefix and R is connected on the prefixes of one list, so the longest exists.

X⊑Y, X entire, Y simple ⇒X=Y

⦇[nil,(π₁p→cons,⊸ nil)]⦈ is a reduce of maps, hence entire — what turns the ∃ above into the heading's =.

§7.2.5. mss = ⦇k⦈ π₂ est(≥) — mss is a fold followed by taking the largest

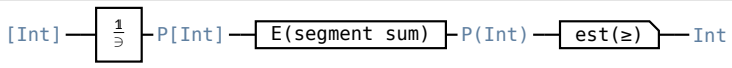
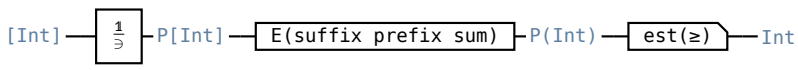
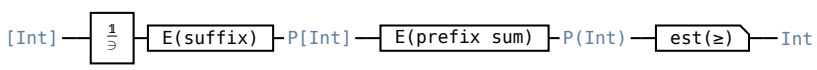
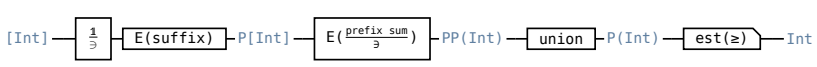
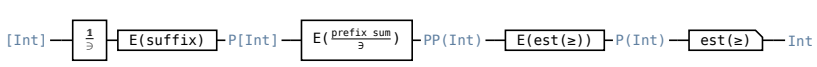
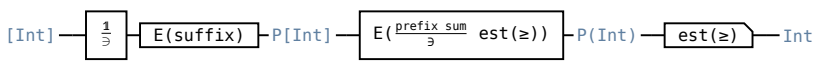
(7.2.5a)

⊕ ≜ $\frac{\text{⊸ zero ∪ plus}}{\ni}$ est(≥) — ⊕ is the larger of 0 and a+b; B&dM write max(Λ(zero ∪ plus)).

⊕≜ $\frac{\text{⊸ zero ∪ plus}}{\ni}$ est(≥) FX=𝟙+Int×X, α≜[nil,cons], head≜cons° π₁, τ≜⟨𝟙,⊸ nil⟩ cons.

mss = $\frac{\text{suffix}}{\ni}$ E($\frac{\text{prefix sum}}{\ni}$ est(≥)) est(≥) (7.2.5b)

maximum segment sum problem: using **segment=suffix prefix**, the specification is expressed in this form

formula — one wire from `[Int]` to `Int`, its type written along it	**reason**
	
= 	segment=suffix prefix segment = suffix prefix , ≜ mss
= 	absorption $\frac{S}{\ni}$ E(R) = $\frac{S\ R}{\ni}$ — frac(S,∋) E(R)=frac(SR,∋) at S:=suffix, R:=prefix sum
= 	$\frac{R}{\ni}$ ∋=R, union=E(∋) $\frac{R}{\ni}$ ∋ = R frac(R,∋)∋=R at R:=prefix sum and E(R)≜frac(∋R,∋), E(R) ≜ $\frac{\ni\ R}{\ni}$; union ≜ $\frac{\ni\ \ni}{\ni}$ union≜frac(∋∋,∋); the middle equality is ≜ relator's F(RS)=F(R)F(S) at F:=E
= 	R² ⊑ R ⇒ P(est(R)) est(R) = P(dom(est(R))) union est(R) , the sets non-empty
= 	relator ≜ relator — F(RS)=F(R)F(S) at F:=E

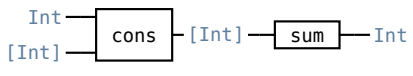
The union step is R² ⊑ R ⇒ P(est(R)) est(R) = P(dom(est(R))) union est(R) 's P(est(R)) est(R)=P(dom(est(R))) union est(R) — every suffix has the empty prefix, so Dom is 𝟙 here — and P(f)=E(f) at the map it is applied to (Map(f) ⇒ P(f) = E(f)).

`[[],cons ∪ ⊸ nil] sum = F(sum) S` (7.2.5c)

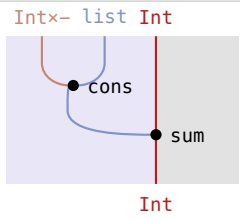
fusion: **`prefix`** expressed as a catamorphism on cons-lists, **`(|[nil,cons ∪ ⊸ nil]|)`**, and this is the fusion condition that expresses **`prefix sum`** as a catamorphism

circuit — the fork is the bracket's case split
`F([Int])=𝟙+Int×[Int]`: `nil` above, the pair and its `∪` below

Hinze–Marsden

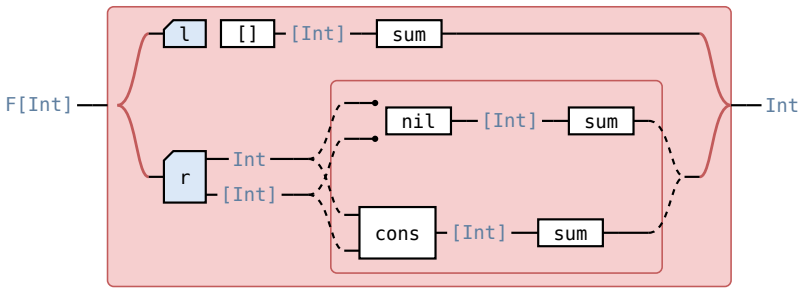


`[[],cons ∪ ⊸ nil] sum`



the `cons` operand of `cons ∪ ⊸ nil`

=



`[[] sum,(⊸ nil sum) ∪ (cons sum)]`

coproduct of maps, composition over `∪`, `∪` commutative

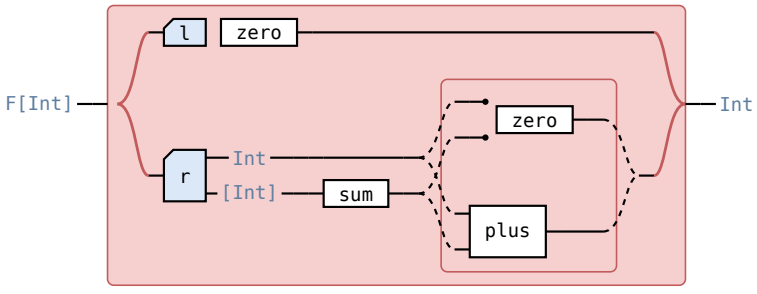
=

`[zero,⊸ zero ∪ ((𝟙 × sum) plus)]`

`sum`'s defining equation

the `(𝟙×sum) plus` operand of `⊸ zero ∪ (𝟙×sum) plus`

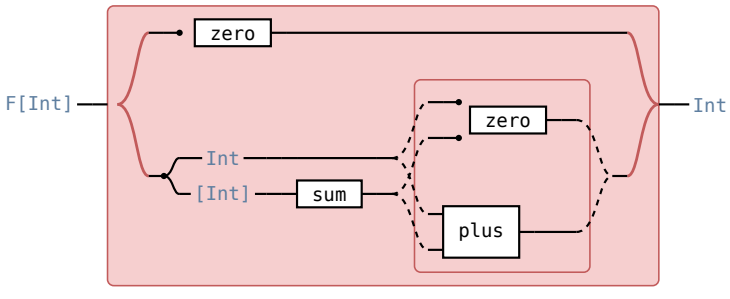
=



`[zero,(𝟙 × sum) (⊸ zero ∪ plus)]`

`(𝟙×sum)⊸=⊸`, `sum` entire

=



`F(sum) [zero,⊸ zero ∪ plus]`

relator

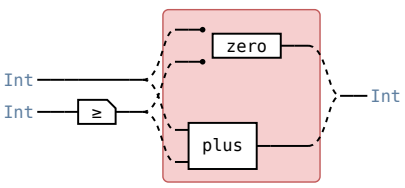
fusion at `R:=[nil,cons ∪ ⊸ nil]`, `S:=sum`: the side condition, so `prefix sum=(|[zero,⊸ zero ∪ plus]|)`. `prefix` is the reduce, `sum` the map fused into it — the intermediate list is gone.

`S Pres ≥` (7.2.5d)

monotonic algebra: an `F`-algebra `S` is monotonic on a relation `R` if `F(R)S⊑SR` — the `plus` branch of `F(≥)S⊑S≥`; the `zero` branch is `zero⊑zero≥`

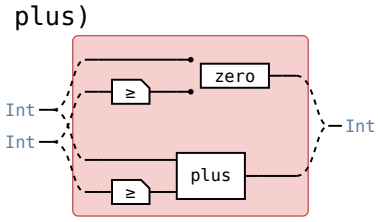
circuit — the head above, the running sum below; the tape is the `∪`

`(𝟙 × ≥) (⊸ zero ∪ plus)`



=

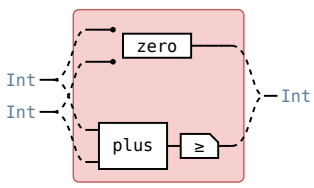
`((𝟙 × ≥) ⊸ zero) ∪ ((𝟙 × ≥) plus)`



relator, composition over `∪`

⊑

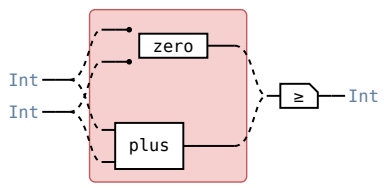
`⊸ zero ∪ (plus ≥)`



`dom(R S) ⊑ dom(R)`, `(≥×≥) plus⊑plus≥`; `(≤×≤) plus⊑plus≤` is ≜ monotonic, written `+` there, and `plus` is a map, so it is monotonic on an order and on its opposite together, which carries it to `≥`.

⊑

`(⊸ zero ∪ plus) ≥`

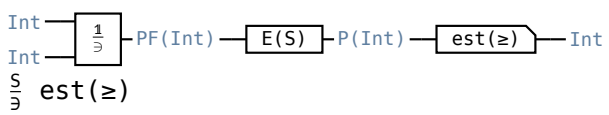


`≥` reflexive

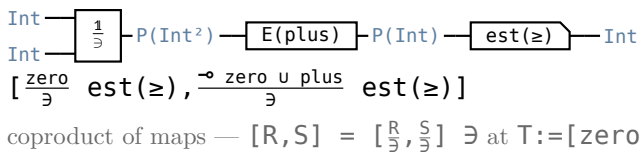
`S/∋ est(≥) = [zero,⊕]` (7.2.5e)

the largest sum the algebra offers is zero from nothing and, from a head and a running sum, the larger of zero and the head added to it

circuit — the tape is the coproduct: `zero`'s branch above, `plus`'s below

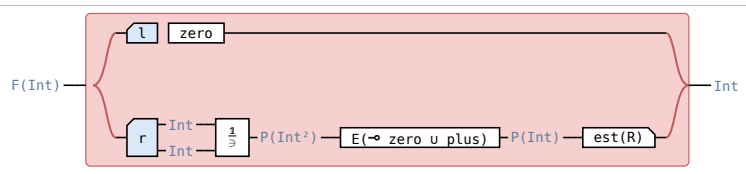


=



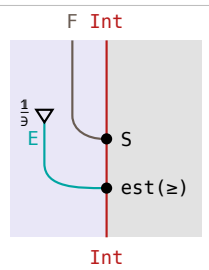
coproduct of maps — `[R,S] = [R/∋,S/∋] ∋` at `T:=[zero,⊸ zero ∪ plus]`, then `[U,V]Z=[UZ,VZ]` — `[R,S] Z = [R Z,S Z]`, composition over `∪`

=

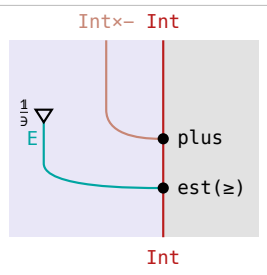


singleton, `≥` reflexive — `1/∋ est(R) = 𝟙 ∩ R°` 's `1/∋ est(R)=𝟙∩R` at `R:=≥`, `zero` a map; the lower branch is `⊕`'s definition, ≜ `mss`, and no law

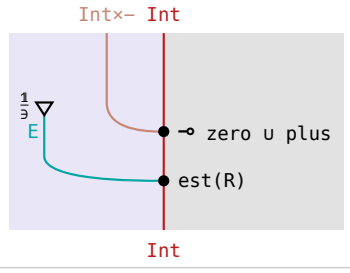
Hinze–Marsden



the `zero` arm of the bracket



the `plus` operand of the lower arm's `⊸ zero ∪ plus`, under its `1/∋ E(…)` and `est(≥)`



with `S Pres ≥` the greedy theorem gives `(|[zero,⊕]|)⊑ prefix sum/∋ est(≥)` — and Exercise 7.40 `mss = (|k|) π₂ est(≥)` makes it an equality.

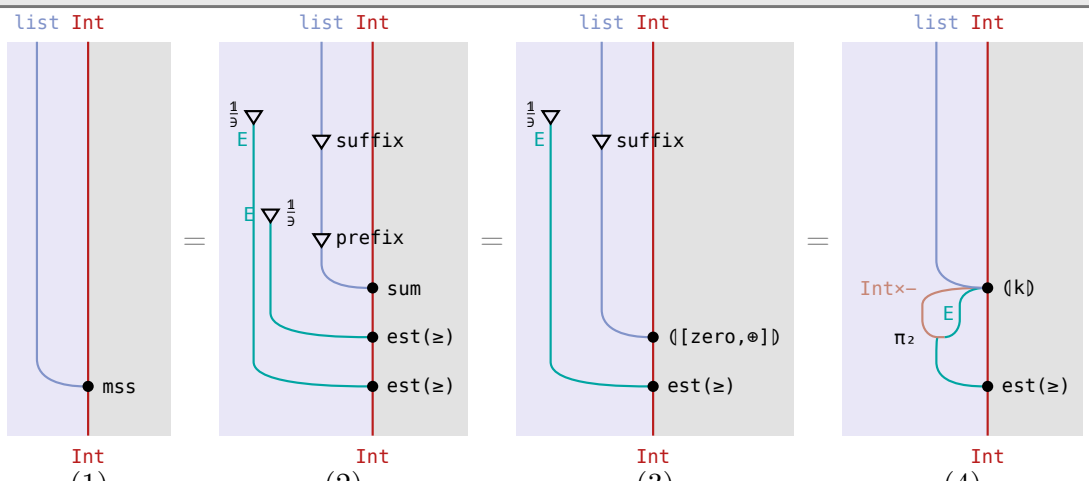
`[[],cons] (◭[zero,⊕]◮, suffix/∋ E(◭[zero,⊕]◮)) = F((◭[zero,⊕]◮, suffix/∋ E(◭[zero,⊕]◮))) k` (7.2.5f)

`k≜[zero⟨𝟙,𝟙/∋⟩,⟨w,⟨w 𝟙/∋,π₂π₂⟩ cup⟩]`, `w≜(𝟙×π₁)⊕`: the value at the whole list, paired with the set of the values at its suffixes, runs `k`'s recursion.

the equation at that branch	why
`nil⟨g, suffix/∋ E(g)⟩=zero⟨𝟙,𝟙/∋⟩`	`nil` has one suffix, itself, and `g(nil)=zero`, so the set is the singleton `{zero}`
`cons⟨g, suffix/∋ E(g)⟩=(𝟙×⟨g, suffix/∋ E(g)⟩)⟨w,⟨w 𝟙/∋,π₂π₂⟩ cup⟩`	`g(cons(a,x))=a⊕(g(x))`, which is `w` reading `g(x)` off `π₁`; and the suffixes of `cons(a,x)` are `cons(a,x)` itself, whose value is that same `w`, together with those of `x`, which `π₂π₂` carries — so the two sets meet at `cup`

`mss = ◭k◮ π₂ est(≥)` Exercise 7.40

maximum segment sum problem: `suffix/∋ E(◭[zero,⊕]◮)` expressed as the catamorphism `◭k◮`, hence `mss` implemented by a linear-time algorithm, `⊕≜ ⊸ zero ∪ plus/∋ est(≥)` — `≜ mss`; `k` and `w` — `[[],cons] ⟨◭[zero,⊕]◮, suffix/∋ E(◭[zero,⊕]◮)) = F((◭[zero,⊕]◮, suffix/∋ E(◭[zero,⊕]◮)))` `k`.



1	`mss`	(7.2.5b) `mss = suffix/∋ E(prefix sum/∋ est(≥)) est(≥)`
2	`= suffix/∋ E(prefix sum/∋ est(≥)) est(≥)`	`prefix sum/∋ est(≥) = ◭[zero,⊕]◮`
3	`= suffix/∋ E(◭[zero,⊕]◮) est(≥)`	`◭k◮ π₂ = suffix/∋ E(◭[zero,⊕]◮)`
4	`= ◭k◮ π₂ est(≥)`	

one fold builds the `n+1` running maxima and the final `est(≥)` reads them in one more pass, so `mss` is linear.

§7.2.6. `Coreflexive(p) ⇒ filter(p) = ⦇[nil,fStep(p)]⦈` — `filter(p)` keeps the items satisfying `p`, as one fold

(7.2.6a)

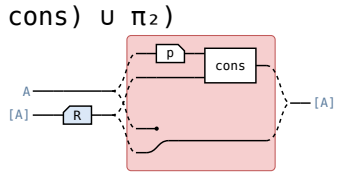
	definition	type	example and in words
1	`FX = L + E × X`	`𝒜→𝒜`	nothing, or a head and a tail
2	`α ≜ [nil,cons]`	`Alg(F,[A])`	both constructors as one map
3	`p ⊑ 𝟙`	`A→A`	`p≜even` — `2 p 2`, and `3∉dom(p)` — a coreflexive: `a` passes `p` when `a p a`
4	`R ≜ length ≤ length°`	`[A]→[A]`	`[1] R [1,2]` — `xs R ys⇔length(xs)≤length(ys)`
5	`π₂(a,x) ≜ x`	`A×[A]→[A]`	`π₂(3,[1,2])=[1,2]` — drop the head, keep the tail
6	`subseq = ⦇[[],cons ∪ π₂]⦈`	`[A]→[A]`	`[3,1,2] subseq [3,2]` — `xs subseq ys⇔ys` is `xs` with elements dropped — at each `cons`, keep the head or drop it
7	`S ≜ [[],((p × 𝟙) cons) ∪ π₂]`	`Alg(F,[A])`	`(4,[2]) S [4,2]` and `(4,[2]) S [2]`, but `(3,[2]) S [2]` only — `subseq`'s algebra with one extra `p` — keep a head that passes `p`, or drop the head
8	`𝟙⊑π₂R cons°`		the tail is one shorter than the cons, so `π₂` loses the `est(R°)` at every step — where `(a,xs) ↦ []` 's loser is `nil`

`Coreflexive(p)`
`⇒ S Pres R°` (7.2.6b)

shortening the tail and then taking the step lands inside taking the step and then shortening the result

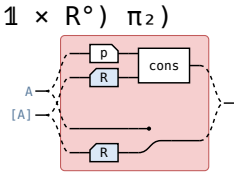
formula — the `cons` branch of `F(R°)S⊑SR°`; **reason** under each circuit

`(𝟙 × R°) (((p × 𝟙) cons) ∪ π₂)`



=

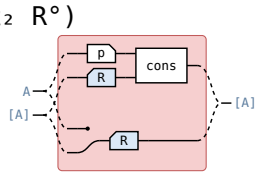
`((p × R°) cons) ∪ ((𝟙 × R°) π₂)`



`(𝟙×R°)(p×𝟙)=p×R°` relator

=

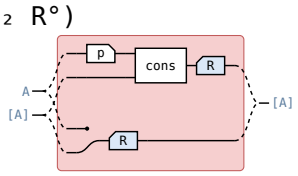
`((p × R°) cons) ∪ (π₂ R°)`



≜ `(𝟙×R°)π₂=π₂R°` (`𝟙 × ∋) π₂ = π₂ ∋`

⊑

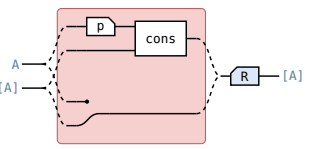
`((p × 𝟙) cons R°) ∪ (π₂ R°)`



`(p×R°) cons⊑(p×𝟙) cons R°`
`Coreflexive(p) ⇒ (𝟙 × R°) (((p × 𝟙) cons) ∪ ⊸ nil) ⊑ (((p × 𝟙) cons) ∪ ⊸ nil) R°`

⊑

`(((p × 𝟙) cons) ∪ π₂) R°`



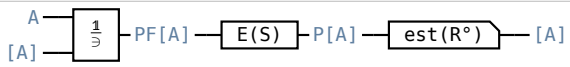
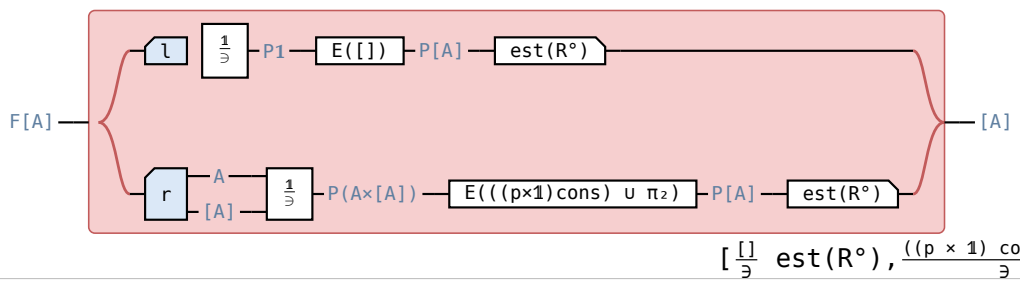
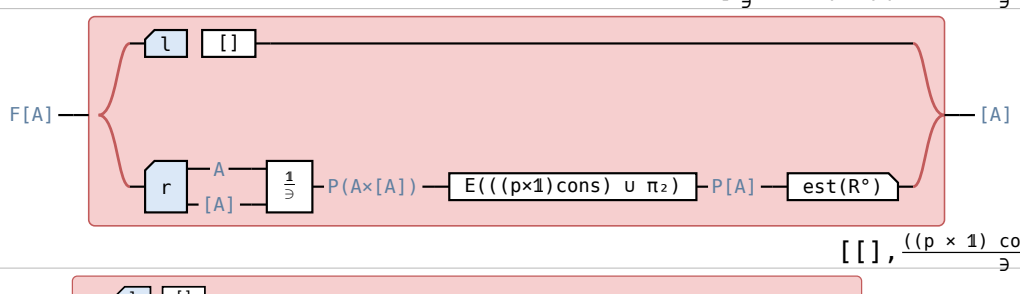
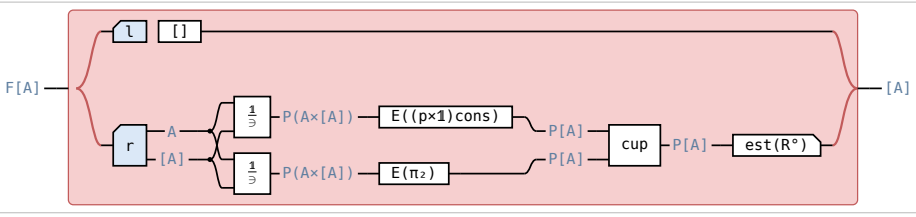
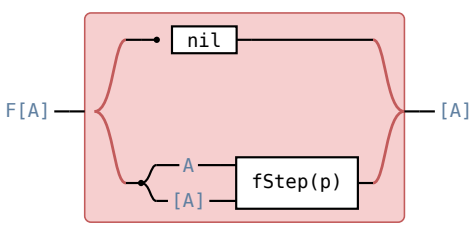
`(S ∪ T) R = (S R) ∪ (T R)` , `(R ∩ R') / S = (R / S) ∩ (R' / S)`

the `nil` branch: `nil⊑nil R°`.

Coreflexive(p) (7.2.6c)

⇒ $\frac{S}{\ni}$ est(R°) = [nil,fStep(p)]

the longest of the lists the algebra allows is the cons where the head passes p, and the tail where it does not

formula	reason
	
= 	S ≜ [[],((p × 𝟙) cons) ∪ π₂] — and the fraction of a coproduct of maps is the coproduct of their fractions [R,S] = [$\frac{R}{\ni}$,$\frac{S}{\ni}$] ∋
= 	$\frac{nil}{\ni}$ is the singleton {nil}, and est(R°) of a singleton is its element because R° is reflexive ≜ est
= 	$\frac{(p×𝟙)\ cons\ ∪\ π₂}{\ni}$ = ⟨$\frac{(p×𝟙)\ cons}{\ni}$, $\frac{π₂}{\ni}$⟩ cup ≜ cup
= 	𝟙⊑π₂R cons° (7.2.6a.8) — xs R cons(a,xs), so est(R°) returns the cons where p a puts it in the set and xs where the set is {xs} ≜ est

the head is dropped, not the whole tail: that is the one place π₂ shows against Coreflexive(p)

⇒ $\frac{S}{\ni}$ est(R°) = [nil,twStep(p)] 's ⊸ nil.

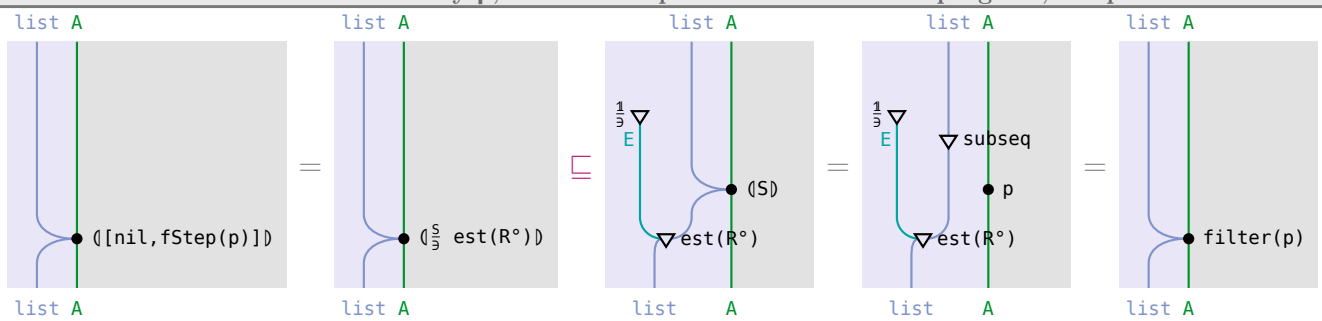
(7.2.6d)

definition	type	example and in words
S ≜ [[],((p × 𝟙) cons) ∪ π₂]	Alg(F,[A])	(4,[2]) S [4,2] and (4,[2]) S [2], but (3,[2]) S [2] only — subseq's algebra with one extra p — keep a head that passes p, or drop the head

Coreflexive(p) Exercise 7.41

⇒ filter(p) = ⦇[nil,fStep(p)]⦈

filter: **filter(p) x** returns the longest subsequence of **x** with the property that all its elements satisfy **p**; the catamorphism is the standard program; **R** a preorder.



1	⦇[nil,fStep(p)]⦈	(7.2.6c) Coreflexive(p) ⇒ ⫕ est(R°) = [nil,fStep(p)]
2	= ⦇⫕ est(R°)⦈	Coreflexive(p) ⇒ ⦇⫕ est(R°)⦈ ⊑ ⦇S⦈/∋ est(R°)
3	⊑ ⦇S⦈/∋ est(R°)	Coreflexive(p) ⇒ subseq p′ = ⦇S⦈
4	= subseq p′/∋ est(R°)	
5	= filter(p)	

the catamorphism is entire and `filter(p)` simple, so ⊑ is =.

`(subseq p′)°(subseq p′)∩R∩R°⊑𝟙` fails — two `p`-subsequences of one list can be of equal length and different — so §Section 7.2.4.'s uniqueness argument does not transfer. What survives `est(R°)` is the one keeping **exactly** the passing elements: a subsequence that drops a passing element is beaten by the one that keeps it.

(7.2.6e)

	definition	type	note
1	segment = suffix prefix	[A]→[A]	A contiguous stretch of `xs`: a suffix, then a prefix of that.
2	filter(p)≜ subseq p′/∋ est(R°)	[A]→[A]	The longest subsequence of `xs` whose every element passes `p`. `est(R°)` is ≜ `est`
3	R≜length≤length°	[A]→[A]	The preorder `filter` and `takewhile` maximise over: the longer list wins. ≥≜≤°
4	takewhile(p)≜ prefix p′/∋ est(R°)	[A]→[A]	The same with `prefix` for `subseq`: the longest prefix whose every element passes `p`.
5	mss≜ segment sum/∋ est(≥)	[Int]→Int	Maximum segment sum. `segment=suffix prefix` splits it into prefix sum/∋ `est(≥)` on each suffix.

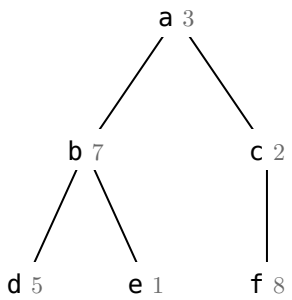
§7.3. Planning a company party

(7.3a)

	definition	type	note
1	`tree A::=node (A,[tree A])`	$\mathcal{A}\to\mathcal{A}$	The company hierarchy: an employee, and the list of subtrees under them.
2	`F(A,B)=A×[B]`	$\mathcal{A}\to\mathcal{A}$	The base functor `tree` folds: an employee beside the recursive position, one layer deep.
3	`rating`	`A→Int`	What one employee is worth as a guest.
4	`cost = rating´ sum`	`[A]→Int`	What a guest list is worth.
5	`R = cost ≤ cost°`	`[A]→[A]`	The preorder the guest list is maximised over.
6	`choose ≜ π₁ ∪ π₂`	`[A]×[A]→[A]`	Takes one of the two parties a subtree returns.
7	`include = (𝟙 × (π₂´ concat)) cons`	`F(A,[A]×[A])→[A]`	The party that invites the root, which puts every immediate subtree's root out. A map.
8	`exclude = (𝟙 × (choose´ concat)) π₂`	`F(A,[A]×[A])→[A]`	The party that leaves the root out, so each subtree is free to choose. Not a map.
9	`S ≜ ⟨include,exclude⟩`	`F(A,[A]×[A])→[A]×[A]`	The algebra: one step returns both parties of a subtree at once.
10	`party = (⦇S⦈) choose`	`tree(A)→[A]`	Every guest list the president's ruling allows.
11	the specification $\frac{\texttt{party}}{\ni}$ `est(R°)`	`TA→[A]`	A guest list of greatest total conviviality.

§7.3.1. `include` and `exclude`

(7.3.1a)



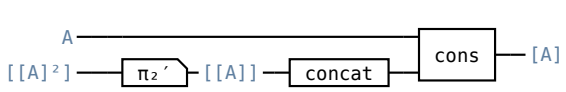
the small grey number is the employee's rating

(7.3.1b)

	node	the algebra's input `A×[[A]×[A]]`	`include`	`exclude`
1	`d`, `e`, `f`	`(d,[])`	`[d]`	`[]`
2	`b`	`(b,[([d],[]),([e],[])])`	`[b]`	`[d,e]`, `[d]`, `[e]`, `[]`
3	`c`	`(c,[([f],[])])`	`[c]`	`[f]`, `[]`
4	`a`	`(a,[([b],…),([c],…)])`	`[a]#exclude(b)#exclude(c)`	`choose(b)#choose(c)`

with those ratings, `est(R°)` keeps `([b]=7,[d,e]=6)` at `b`, where `include` wins, and `([c]=2,[f]=8)` at `c`, where `exclude` wins, so at the root `include=[a,d,e,f]=17` beats `exclude=[b,f]=15`, and `choose` takes 17.

(7.3.1c)



(7.3.1d)



(7.3.1e)

```
choose´ [([d],[]),([e],[])]
  1st item              ([d],[])  choose↦[d]  or  []
  2nd item              ([e],[])  choose↦[e]  or  []
  2×2=4 combinations:   [[d],[e]]   [[d],[]]   [[],[e]]   [[],[]]
  concat flattens:        [d,e]       [d]        [e]        []
```

§7.3.2. `(R×R)°´`

(7.3.2a)

```
[  ([b],[d,e])  ,  ([c],[f])  ]
        │              │
        ▼              ▼        (R×R)°´, elementwise
[   ([b],[d])   ,   ([c],[])   ]
```

```
([b],[d,e])
  │   └── exclude: the best party in b's subtree when b does not come
  └─────── include: the best party in b's subtree when b comes
```

(7.3.2b)

	element	1st component include	2nd component exclude
1	1st, `([b],[d,e])`	`[b] R° [b]` 7≥7, reflexivity	`[d,e] R° [d]` 6≥5, and `([b],[d,e])` is the pair `est(R°)` keeps at `b`
2	2nd, `([c],[f])`	`[c] R° [c]` 2≥2, reflexivity	`[f] R° []` 8≥0, and `([c],[f])` is the pair `est(R°)` keeps at `c`

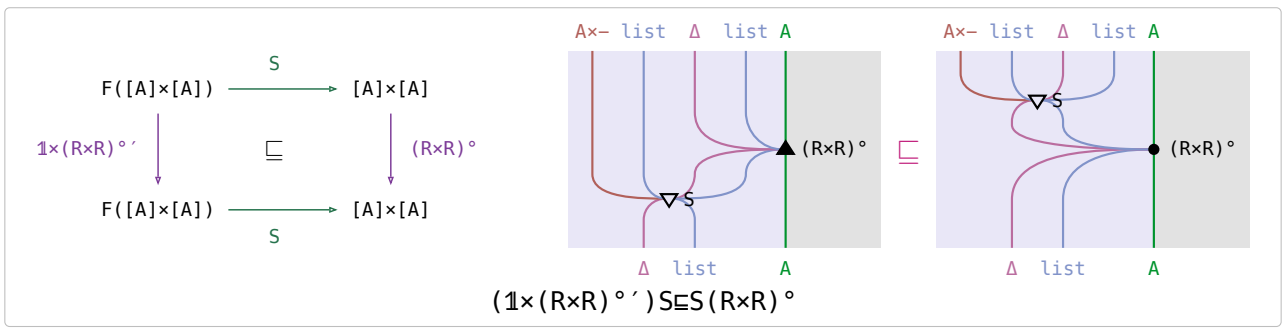
the first component of `𝟙×(R×R)°´` is the root employee.

`R≜cost≤cost°` is a preorder, so the components that do not move are related by reflexivity — `(R×R)°` demands a relation in **both** components, it cannot skip one.

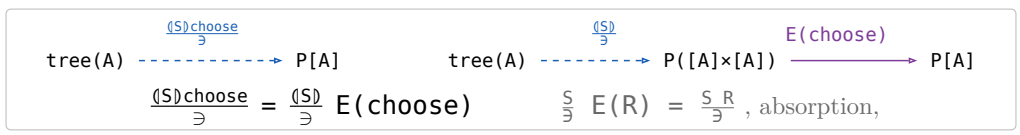
R compares cost only, not contents: `[f] R° []` is legal though `[f]` has an element `[]` does not. That is why the three leaves of §Section 7.3.3.'s proof all reduce to "`cost` is a sum".

§7.3.3. `(𝟙 × (R × R)°´) S ⊑ S (R × R)°` — S is monotonic on `(R×R)°`

(7.3.3a)



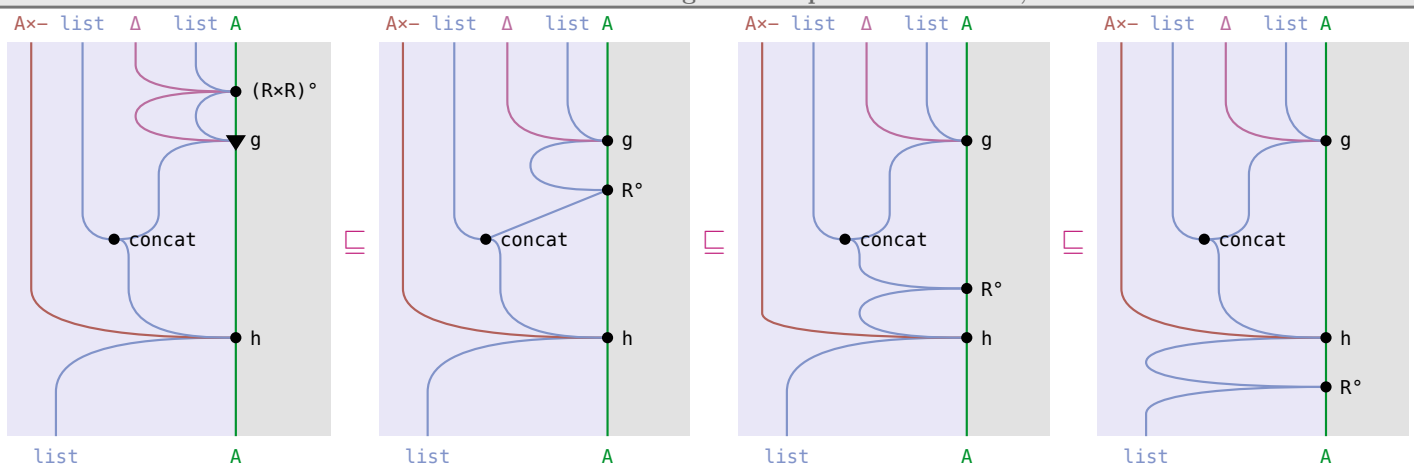
(7.3.3b)



S ≜ ⟨include,exclude⟩ **(7.3.3c)**

(𝟙 × (R × R)°′) include ⊑ include R°
(𝟙 × (R × R)°′) exclude ⊑ exclude R°
(𝟙 × (R × R)°′) S ⊑ S (R × R)°

bettering both parties of every subtree before the node's algebra runs gets no further than running it first and bettering the two parties it returns,



(1) (2) (3) (4)

1	(𝟙 × (R × R)°′) (𝟙 × (g′ concat)) h	(R × R)° g ⊑ g R°
2	⊑ (𝟙 × ((g R°)′ concat)) h	R°′ concat ⊑ concat R°
3	⊑ (𝟙 × (g′ concat R°)) h	(𝟙 × R°) h ⊑ h R°
4	⊑ (𝟙 × (g′ concat)) h R°	

g	g:=π₂ is (R×R)°π₂=(dom(π₁R°))π₂R°⊑π₂R° , g:=π₁ its mirror (dom(π₂R°))π₁R°⊑π₁R° — ≜ × and ⟨R,S⟩ π₁ = dom(S) R , ⟨R,S⟩ π₂ = dom(R) S , then dom⊑𝟙; g:=choose≜π₁ ∪ π₂ is the union of the two lax(φ); lax(ψ) ⇒ G(R) (φ_B ∪ ψ_B) ⊑ (φ_A ∪ ψ_A) F(R) . list monotonic, ≜ relator
concat	a LEAF: no law above it. cost is a sum, so cost(concat(xss))=sum(cost′(xss)) and a cheaper part makes a cheaper whole.
h	a LEAF: cost(cons(a,xs)) =rating(a)+cost(xs), so a cheaper tail makes a cheaper list. For h:=π₂ it is an EQUALITY (𝟙×R°)π₂=(dom(π₁))π₂R° =π₂R° : π₁ is a map, hence entire, so dom(π₁)=𝟙 — Entire(R) ≜ dom(R) = 𝟙

(𝟙×(g′ concat))h	g	h
include	π₂	cons
exclude	choose	π₂

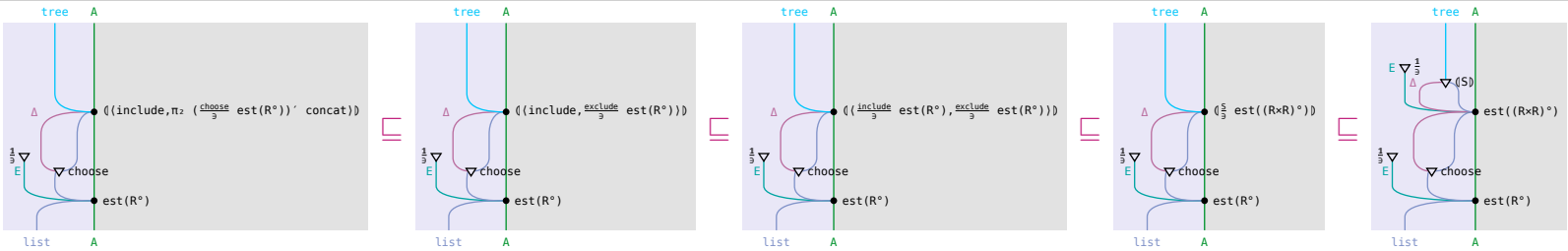
(7.3.3d)

1	R≜{(0,0)} : {0,1}→{0,1}	not entire — undefined at 1
2	(R×R)choose at (0,1)	nothing — R×R needs BOTH components related, and R is undefined at 1
3	choose R at (0,1)	0 — choose picks the first component 0, and R 0 0 holds
4	((0,1),0)	in choose R, not in (R×R)choose
5	(R×R)choose=choose R	iff R is entire — dom⊑𝟙 is the g row's only inequality step, and it is an equality iff dom(R)=𝟙

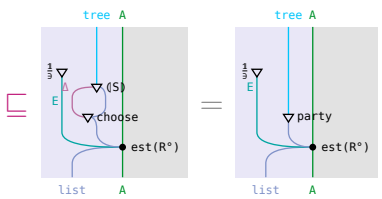
§7.3.4. Derivation

$$(\!|\texttt{include},\pi_2\ (\overset{\texttt{choose}}{\ni}\ \texttt{est(R°)})'\ \texttt{concat}|\!)\ \overset{\texttt{choose}}{\ni}\ \texttt{est(R°)} \sqsubseteq \overset{\texttt{party}}{\ni}\ \texttt{est(R°)} \qquad (7.3.4a)$$

the best of every guest list the president allows is one pass up the tree, each subtree handing up its best party with its boss in and its best with the boss out, and **choose** taking the better of the two at the root



1	$(\!\|\texttt{include},\pi_2\ (\overset{\texttt{choose}}{\ni}\ \texttt{est(R°)})'\ \texttt{concat}\|\!)\ \overset{\texttt{choose}}{\ni}\ \texttt{est(R°)}$	$\pi_2\ (\overset{\texttt{choose}}{\ni}\ \texttt{est(R°)})'\ \texttt{concat} \sqsubseteq \overset{\texttt{exclude}}{\ni}\ \texttt{est(R°)}$
2	$\sqsubseteq (\!\|\texttt{include}, \overset{\texttt{exclude}}{\ni}\ \texttt{est(R°)}\|\!)\ \overset{\texttt{choose}}{\ni}\ \texttt{est(R°)}$	$\texttt{include} \sqsubseteq \overset{\texttt{include}}{\ni}\ \texttt{est(R°)}$
3	$\sqsubseteq (\!\|\langle \overset{\texttt{include}}{\ni}\ \texttt{est(R°)}, \overset{\texttt{exclude}}{\ni}\ \texttt{est(R°)}\rangle\|\!)\ \overset{\texttt{choose}}{\ni}\ \texttt{est(R°)}$	$\langle \overset{\texttt{include}}{\ni}\ \texttt{est(R°)}, \overset{\texttt{exclude}}{\ni}\ \texttt{est(R°)}\rangle \sqsubseteq \overset{\mathbb{S}}{\ni}\ \texttt{est((R × R)°)}$
4	$\sqsubseteq (\!\|\overset{\mathbb{S}}{\ni}\ \texttt{est((R × R)°)}\|\!)\ \overset{\texttt{choose}}{\ni}\ \texttt{est(R°)}$	$(\!\|\overset{\mathbb{S}}{\ni}\ \texttt{est((R × R)°)}\|\!) \sqsubseteq \overset{(\!\|\mathbb{S}\|\!)}{\ni}\ \texttt{est((R × R)°)}$
5	$\sqsubseteq \overset{(\!\|\mathbb{S}\|\!)}{\ni}\ \texttt{est((R × R)°)}\ \overset{\texttt{choose}}{\ni}\ \texttt{est(R°)}$	$\texttt{est((R × R)°)}\ \overset{\texttt{choose}}{\ni}\ \texttt{est(R°)} \sqsubseteq \texttt{E(choose) est(R°)}$



6	$\sqsubseteq \overset{(\!\|\mathbb{S}\|\!)\ \texttt{choose}}{\ni}\ \texttt{est(R°)}$	$\texttt{party} = (\!\|\mathbb{S}\|\!)\ \texttt{choose}$
7	$= \overset{\texttt{party}}{\ni}\ \texttt{est(R°)}$	

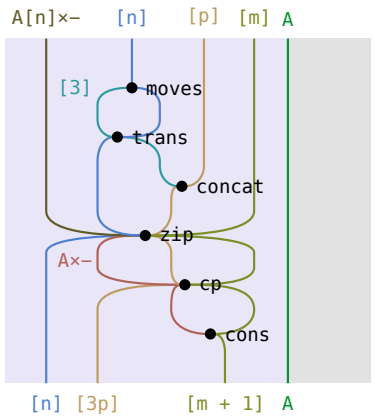
§7.4. Shortest paths on a cylinder

(7.4a)

		on L=list⁺, A.3.3	on Vec	note
1	F	F(A,X)=A+A×X	F(A,A[m])=A[1]+A[m+1]	A square alone, or in front of the path so far.
2	the input	L(N A)	A[m][n]≜(A[n])[m]	m columns of n squares, outermost index first — the matrix itself.
3	a path	L A, a non-empty list	A[1]≜A, A[m+1]≜A×A[m]	Through m columns, m squares: the count is in the type. A one-square path is the square; Vec(m) is the functor X↦X[m].
4	the paths into a row	P(L A)	A[p][m], p=3^(m-1)	Three moves per column crossed, so this count is in the type too; a set forgets it.
5	α	[τ,cons] F(A,LA)→LA	τ=𝟙:A→A[1] cons:A×A[m]→A[m+1]	τ is an identity; cons is only an iso, so it stays a bead.
6	moves	NA→PNA	X[n]→X[3][n]	moves(x)=(up(x),x,down(x)), a 3×n matrix.
7	trans	PNX→NPX	X[3][n]→X[n][3]	The transpose.
8	union	PPX→PX	concat:X[j][k]→X[jk]	The j rows laid end to end; j is 3 inside gen and n inside paths.
9	zip	F(NA,NB)→NF(A,B)	F(A[n],B[n])→F(A,B)[n]	Unchanged.
10	cp	F(A,PB)→P(F(A,B))	A×B[p]→(A×B)[p]	The square paired with each of the p candidates.
11	est(R)≜∋n(∈\R*)X	P*X→X	X[k]→X, ∋:X[k]→X	A component that is R-below every component; ∋ is "is some component" and ∈ its converse.
12	R≜sum≤sum°	[ℕ]→[ℕ]	ℕ[m]→ℕ[m]	The cost of a path, which the cheapest minimises — one order per length m.
13	gen	F(𝟙,moves trans N(union)) zip N(cp P(α)) F(NA,N(P(LA))) →N(P(LA))	F(𝟙,moves trans Vec(n)(concat)) zip Vec(n)(cp Vec(3p)(cons)) F(A[n],A[n][p][m])→A[n][3p][m+1]	The same composite without α; the type shown is the cons side, the τ side is 𝟙:A[n]→A[n].
14	⦇gen⦈	L(N A)→N(P(L A))	A[m][n]→A[n][p][m]	Columns in, rows out.
15	paths≜⦇gen⦈ concat	⦇gen⦈ setify union [A[n]]→P[A]	A[m][n]→A[np][m]	The n rows of p paths laid end to end: every path of the cylinder, in one row.
16	Q, Section 7.4.3.	F(𝟙,moves trans N(est(R))) zip N(α)	F(𝟙,moves trans Vec(n)(est(R))) zip, F(A[n],A[n][m])→A[n][m+1]	The cheapest of three; no p anywhere. ⦇Q⦈:A[m][n]→A[n][m] has the type of a transpose.
17	the specification paths est(R)	[ℕ[n]]→[ℕ]	A[m][n]→A[m], est(R):X[np]→X	A cheapest path from the entry side to the exit side.

§7.4.1. `gen ≜ (𝟙 × (moves trans Vec(concat))) zip Vec(cp Vec(cons))` — one more column: each square's paths go to its three neighbours, then the square goes in front

(7.4.1a)

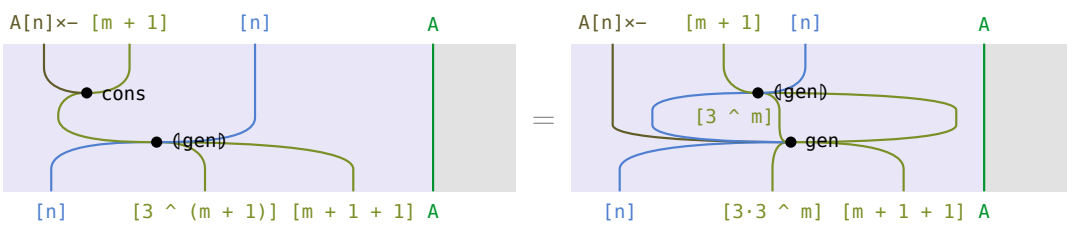


(7.4.1b)

```
a row is the outer index, a cell one path
u = ((1,2,3,4),C)                                  : A[n]×A[n]
    C = 5                                            one candidate per row, one square long:
        6                                            a column of squares
        7
        8
(𝟙×(moves trans Vec(n)(concat)))                     𝟙 keeps the column, the candidates move
  moves(C)                                           down, unmoved, up: a 3×n matrix
   = 6 7 8 5
     5 6 7 8
     8 5 6 7                                       : A[3][n]
  trans(that)                                        the transpose: row k gets rows k-1, k, k+1
   = 6 5 8
     7 6 5
     8 7 6
     5 8 7                                         : A[n][3]
  Vec(n)(concat)(that) = that                        concat:X[3][1]→X[3] is 𝟙
zip(that)                                            each row: its square, and its candidates
     = (1, 6 5 8)
       (2, 7 6 5)
       (3, 8 7 6)
       (4, 5 8 7)                                  : (A×A[3])[n]
Vec(n)(cp)(that)                                     cp pairs the square with each candidate:
   = 1 6   2 7   3 8   4 5                           row k of that is the k-th 3×2 block, left
     1 5   2 6   3 7   4 8                           to right; each line a two-square path
     1 8   2 5   3 6   4 7                         : A[n][3][2]
```

§7.4.2. `cons ⦇gen⦈ = (𝟙 × ⦇gen⦈) gen` — folding a matrix with one more column is folding the rest, then one gen

(7.4.2a)



the `[m+1]` bracket puts the column back on the matrix, then the fold reads all of it

the fold reads the rest under `F`, then one `gen` puts the column in front

(7.4.2b)

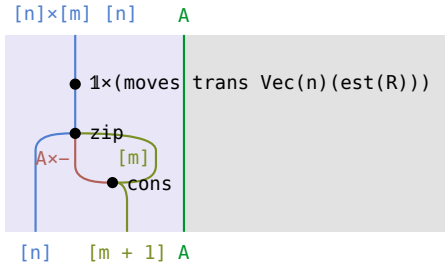
```
xs = 1 2 3 4                                       : A[m][n], m=2
     5 6 7 8

⦇gen⦈(xs) = gen((1,2,3,4), ⦇gen⦈(5,6,7,8))
⦇gen⦈(5,6,7,8)                                       one column: each square its own one-square path
   = 5
     6
     7
     8                                             : A[n][1][1]
```

both halves are in `gen ≜ (𝟙 × (moves trans Vec(concat))) zip Vec(cp Vec(cons))` : the tail folds to the column `5 6 7 8`, one path per row and each of them one square long, and `gen` on it is that walk.

§7.4.3. `Q ≜ (𝟙 × (moves trans Vec(n)(est(R)))) zip Vec(n)(cons)` — Q is gen with est(R) choosing one path per square

(7.4.3a)



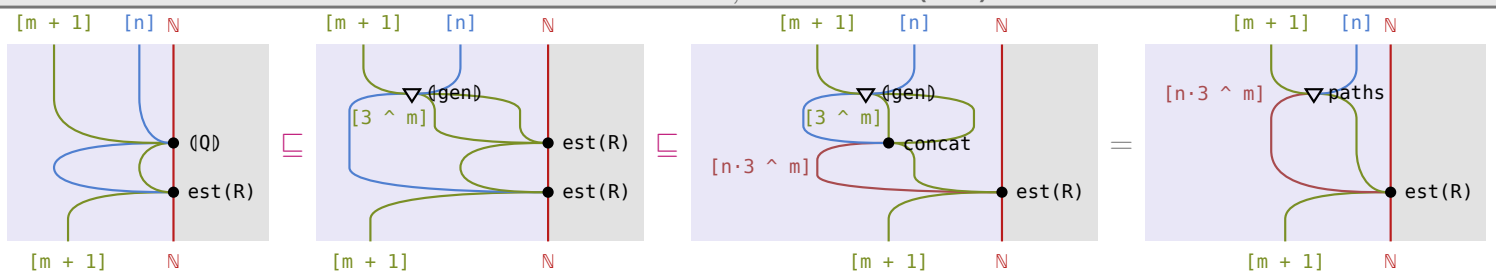
(7.4.3b)

```
u = ((1,2,3,4),C)                          : A[n]×A[n][m]
    C = 5                                    one path per row, one square long
        6
        7
        8
(𝟙×(moves trans Vec(n)(est(R))))
  moves(C)                                   𝟙 keeps the column, the paths move
   = 6 7 8 5                                 down, unmoved, up: a 3×n matrix
     5 6 7 8
     8 5 6 7                               : A[3][n][m]
  trans(that)                                the transpose: row k gets rows k-1, k, k+1
   = 6 5 8
     7 6 5
     8 7 6
     5 8 7                                 : A[n][3][m]
  Vec(n)(est(R))(that)                       the cheapest into each row – chosen BEFORE
   = 5 5 6 5                               : A[n][m]   the new square is put in front
zip(that)                                    each row: its square, and the one predecessor
   = 1 5                                     that survived; the `[m+1]` bracket is cons
     2 5
     3 6
     4 5                                   : A[n][m+1]
```

§7.4.4. `(Q) est(R) ⊑ paths est(R)` — folding Q gives a smallest of paths

`(Q) est(R) ⊑ paths est(R)` (7.4.4a)

a cheapest of all **np** paths of the cylinder is beaten by the greedy fold's one path per row and then a cheapest of those **n**, which costs **O(n×m)**.



1	`(Q) est(R)`	`(Q) ⊑ (gen) Vec(n)(est(R))`
2	`⊑ (gen) Vec(n)(est(R)) est(R)`	`R² ⊑ R`
3	`⊑ (gen) concat est(R)`	`f Vec.concat = f Vec.concat`
4	`= paths est(R)`	

`S² ⊑ S`

`⇒ Vec(j)(est(S)) est(S) ⊑ concat est(S)` , R transitive

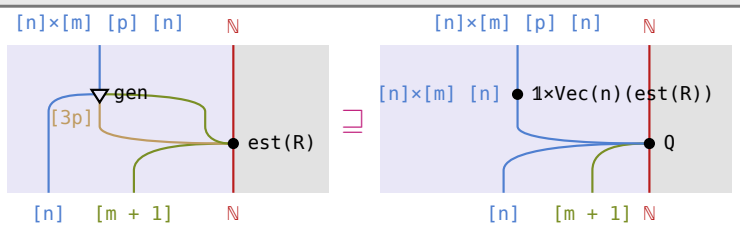
a cheapest of each of the **j** rows and then a cheapest of those **j** is a cheapest of all **jk** entries laid end to end.

`(Q) ⊑ (gen) Vec(n)(est(R))`

the one path per row the greedy fold keeps is one of the **p** that **(gen)** generates for that row, and a cheapest of them.

(𝟙 × Vec(n)(est(R))) Q ⊑ gen Vec(n)(est(R)) (7.13)

choosing a cheapest of each square's **p** paths before the column is extended is no better than extending first and choosing among the **3p**.



1	`gen Vec(n)(est(R))`	(7.13); `zip`, `trans`, `moves` lax natural
2	⊒ `(𝟙 × Vec(n)(est(R))) Q`	

(7.13) on Vec: (𝟙 × est(R)) cons ⊑ cp Vec(k)(cons) est(R)

putting the new square in front of every one of the **p** candidates and then choosing a cheapest is beaten by choosing a cheapest first and putting the square in front of that one.

(7.4.4b)

	stage	type	value	meaning
1	input	`[[ℕ]]`	`[[1, 2, 3, 4], [5, 6, 7, 8]]`	the two columns, square by square
2	`kept(m)`	`[[ℕ]]`	`[[1, 5], [2, 5], [3, 6], [4, 5]]`	square 3 keeps `[3,6]`: 6 is the cheapest of 8, 7, 6
3	`cyl(m,v)`	`[[ℕ]]`	`[[1, 5]]`	the one of cost 6, `est(R)` of the four kept paths

§7.5. The security van problem

(7.5a)

	definition	type	example
1	`R≜length≤length°`	`[[A]]→[[A]]`	`[[a,b,c]]` to `[[a],[b,c]]`, and not back: `1≤2`.
2	`ceiling≜` $\frac{\texttt{prefix sum}}{\ni}$ `est(≥)`	`[A]→Int`	`ceiling[a,b]` is the largest of `0`, `a`, `a+b`.
3	`floor≜` $\frac{\texttt{prefix sum}}{\ni}$ `est(≤)`	`[A]→Int`	`floor[a,b]` is the smallest of `0`, `a`, `a+b`, so `[a,b]` carries `ceiling−floor` cash.

4	`secure`
	the coreflexive on `x` with
	`bmax(ceiling x,ceiling x−floor x)≤N`
		`[A]→[A]`	`secure` keeps `[a]` exactly when `−N≤a≤N`.

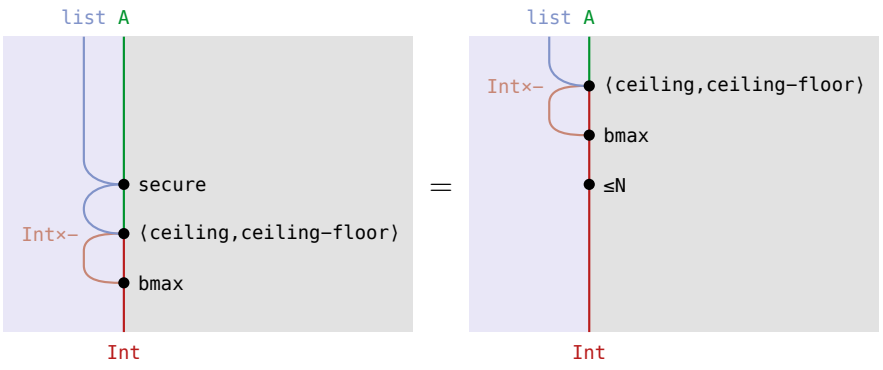
5	`ok` the coreflexive on `(a,xs)` with `xs` non-empty and `[a]#head(xs)` secure	`A×[[A]]→A×[[A]]`	`ok` keeps `(a,[[b],[c]])` exactly when `[a,b]` is secure.
6	`new≜(τ×𝟙) cons`	`A×[[A]]→[[A]]`	`new(a,[[b],[c]])=[[a],[b],[c]]`.
7	`glue≜(𝟙×cons°) assocl (cons×𝟙) cons`	`A×[[A]]→[[A]]`	`glue(a,[[b],[c]])=[[a,b],[c]]`.
8	`old≜(𝟙×cons°) assocl` `((cons secure)×𝟙) cons`	`A×[[A]]→[[A]]`	`old(a,[[b],[c]])=[[a,b],[c]]` when `[a,b]` is secure, and nothing otherwise.
9	`partition = (|[[],new ∪ glue]|)` cutting the transactions into non-empty segments is one pass along them that either opens a segment for the transaction in hand or puts it on the front of the segment already open	`[A]→[[A]]`	`partition[a,b]` gives `[[a],[b]]` and `[[a,b]]`.
10	`S≜[nil,new ∪ old]`	`Alg(F,[[A]])`	`S(a,[[b],[c]])` gives `[[a],[b],[c]]`, and `[[a,b],[c]]` when `[a,b]` is secure.

11	`(∀a. secure[a]) ⇒ partition secure′`
	`= (|S|)`
	cutting the transactions into non-empty segments every way and keeping the cuts whose every segment is secure is the one fold whose algebra is `[nil,new ∪ old]`
		`[A]→[[A]]`	Both take `[a,b]` to `[[a],[b]]` — each `[a]` is secure — and to `[[a,b]]` when `[a,b]` is.

12	`H≜(head prefix° head°) ∪ (nil° nil)`	`[[A]]→[[A]]`	`[[a],[b,c]]` to `[[a,b],[c]]`: `[a]` is a prefix of `[a,b]`.
13	`R;H≜R∩(R°⇒H)`	`[[A]]→[[A]]`	`[[a,b,c]]` to `[[a],[b,c]]` by `|R|`, and `[[a],[b,c]]` to `[[a,b],[c]]` by `R∩H`.
14	`R;H = |R| ∪ R ∩ H` `|R|` is the strict part of `R`, strictly fewer segments; `R;H` splits into it and `R∩H`	`[[A]]→[[A]]`	`[[a,b,c]]` to `[[a],[b,c]]` by `|R|`: `1<2`.
15	the specification $\frac{\texttt{partition secure}'}{\ni}$ `est(R)`	`[A]→[[A]]`	`[a,b]` gives `[[a,b]]` when `[a,b]` is secure, and `[[a],[b]]` otherwise.

§7.5.1. `secure prefix ⊑ prefix secure` — a prefix of a secure segment is secure

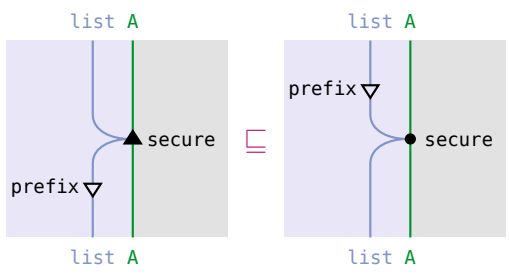
(7.5.1a)



`secure ⟨ceiling,ceiling−floor⟩ bmax = ⟨ceiling,ceiling−floor⟩ bmax (≤N)`

a stretch passes `secure` before the test exactly where the test's own value passes `≤N` after it, which is `secure(xs)` ≜ `bmax(ceiling(xs),ceiling(xs)−floor(xs)) ≤ N` 's `bmax(ceiling x,ceiling x−floor x)≤N`

Exercise 7.49

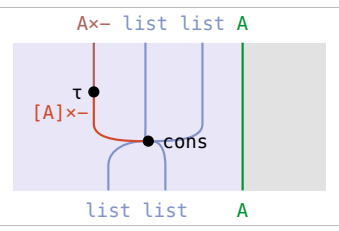
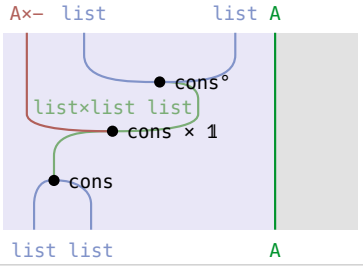
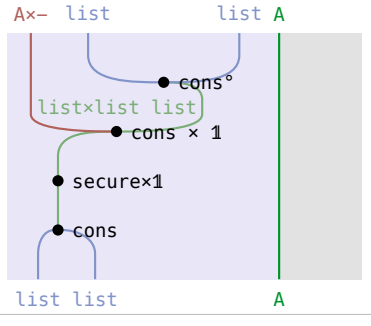


`secure prefix ⊑ prefix secure`

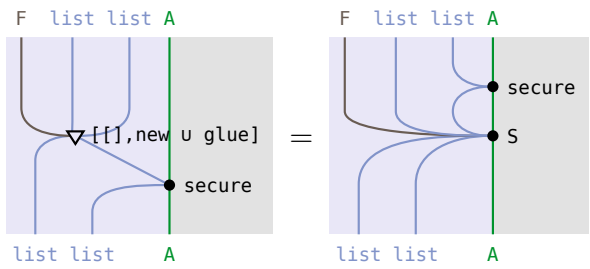
every pair `secure` then `prefix` gives, `prefix` then `secure` gives too — a prefix of a secure stretch is itself secure, which is the prefix-closure B&dM p.185 names

§7.5.2. `(∀a. secure[a]) ⇒ partition secure′ = (𝕊)` — partitioning into secure segments is one fold

(7.5.2a)

	definition	Hinze–Marsden
1	`new≜(τ×𝟙) cons` `τ` makes the transaction a segment on its own, and `cons` puts that segment at the front of the schedule: the van is called	
2	`glue≜(𝟙×cons°) assocl (cons×𝟙) cons` `cons°` splits the schedule into its first segment and the rest, the first `cons` puts the transaction at the front of that segment, and the second puts the segment back	
3	`old≜(𝟙×cons°) assocl ((cons secure)×𝟙) cons` `glue` with `secure` on the segment the transaction has just joined: the van stays away only where the longer segment still passes	

(7.5.2b)



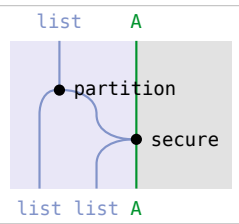
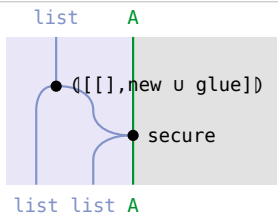
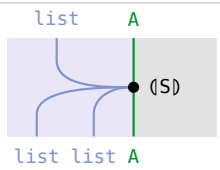
`(∀a. secure[a]) ⇒ [[],new ∪ glue] secure′ = F(secure′) S`

testing every segment for security after the algebra has run is testing it before it ran: a segment `glue` has extended is secure exactly where `old` would have let it be, and the one transaction `new` sets out on its own is secure whatever it is

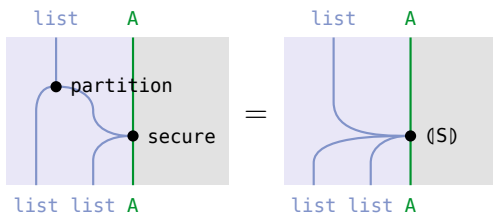
`(∀a. secure[a])`
`⇒ partition secure′ = (|S|)`

Exercise 7.51

cutting the transactions into non-empty segments every way and keeping the cuts whose every segment is secure is the one fold whose algebra is `[nil,new ∪ old]`

step	Hinze–Marsden
`partition secure′` the specification: cut the transactions every way, then keep the cuts whose every segment is secure	
`=  (|[nil,new ∪ glue]|) secure′` `partition` is the fold that either opens a segment for the transaction in hand or puts it on the front of the segment already open	
`=  (|[nil,new ∪ old]|)` fusion: the condition above moves `secure′` inside that fold, which is what turns `glue` into `old`	

(7.5.2c)



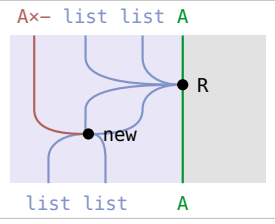
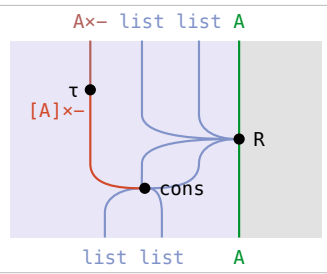
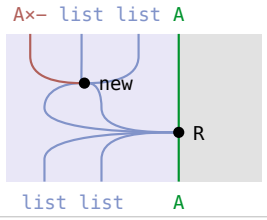
`(∀a. secure[a]) ⇒ partition secure′ = (|S|)`

cutting the transactions every way and then keeping the cuts whose every segment is secure is one pass along them that either calls the van or extends the open segment while it stays secure

§7.5.3. `(𝟙 × R) new ⊑ (new ∪ old) R` — new on an R-related schedule is matched by new or old

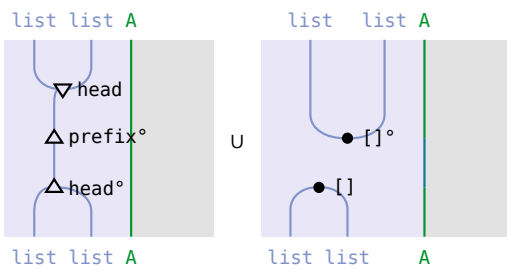
`(𝟙 × R) new ⊑ (new ∪ old) R` (7.14)

calling the van for the transaction on a no-longer schedule gets no further than calling it on this one and shortening the schedule afterwards

step	Hinze–Marsden
`(𝟙×R)new` the left side of (7.14) — `R = length cast ≤ length cast°` , `new(p) ≜ [[π₁(p)]]#π₂(p)`	
`=` `(τ×R)cons` `new≜(τ×𝟙)cons` — `(τ × 𝟙) cons` — and `(τ×𝟙)(𝟙×R)=(τ×R)`	
`⊑ new R` `cons` is monotonic on `R` — `(𝟙×R)cons⊑cons R`, consing onto a no-longer schedule leaves it no longer — and `new⊑new ∪ old`	

§7.5.4. `H ≜ (head prefix° head°) ∪ ([]° [])` — the right first segment is a prefix of the left one, or both are empty

(7.5.4a)

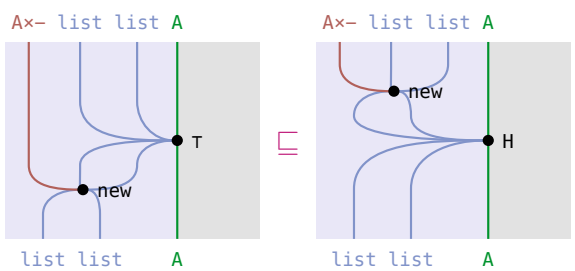


`H ≜ (head prefix° head°) ∪ ([]° [])`

one schedule's first segment is a prefix of the other's, or both schedules are empty

§7.5.5. `(𝟙 × (R;H)) new ⊑ (new ∪ old) (R;H)` — new on an R;H-related schedule is matched by new or old

(7.18)

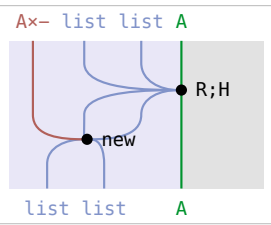
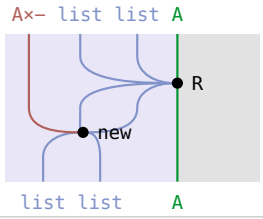
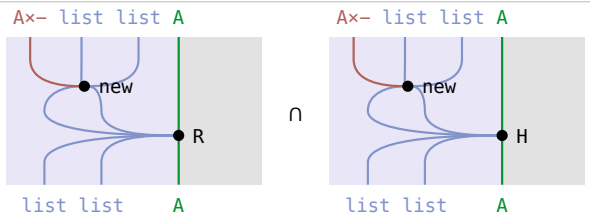
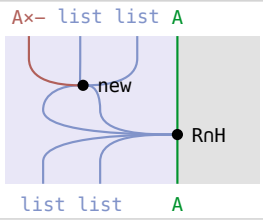
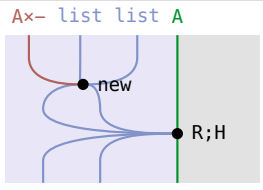


`(𝟙 × τ) new ⊑ new H`

whatever schedule the van is called on, the result's first segment is the one transaction `[a]`, so any two results of `new` on that transaction stand in `H`

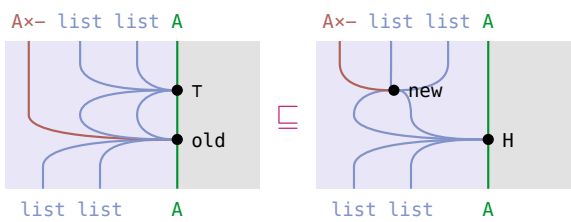
(𝟙 × (R;H)) new ⊑ (new ∪ old) (R;H) (7.16)

calling the van for the transaction on a R;H-better schedule gets no further than calling it on this one and bettering the whole schedule afterwards

step	Hinze–Marsden
(𝟙×(R;H))new the left side of (7.16)	
⊑ (𝟙×R)new R;H⊑R — R;H = R ∩ (R° ⇨ H)	
⊑ new R ∩ new H (7.14) as far as its new R line — (7.14) (𝟙 × R) new ⊑ (new ∪ old) R — and (7.18)	
= new (R∩H) new is a map, and a map distributes over ∩	
⊑ new (R;H) X∩Y⊑X;Y, and new⊑new ∪ old	

§7.5.6. (𝟙 × (R;H)) old ⊑ (new ∪ old) (R;H) — old on an R;H-related schedule is matched by new or old

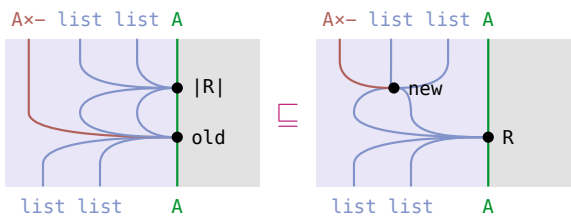
(7.19)



(𝟙 × T) old ⊑ new H

old leaves the transaction [a] at the front of the first segment, and [a] is what new makes that segment, so the two first segments are prefix-related whatever the schedules were

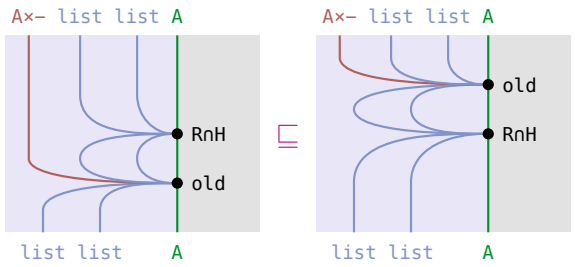
(7.20)



(𝟙 × |R|) old ⊑ new R

old keeps the schedule's length, so a strictly shorter one still comes out no longer than the one the van's own segment lengthens

(7.21)

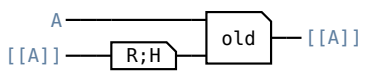
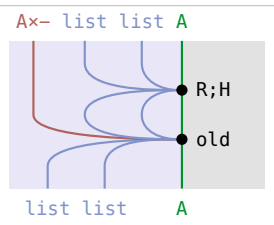
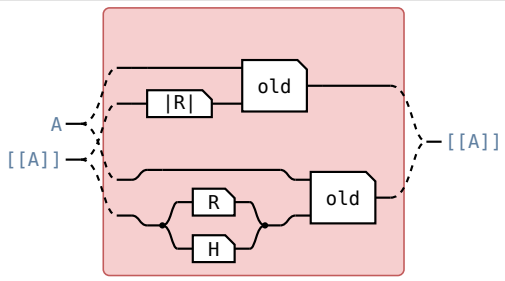
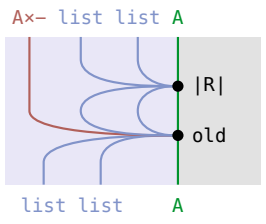
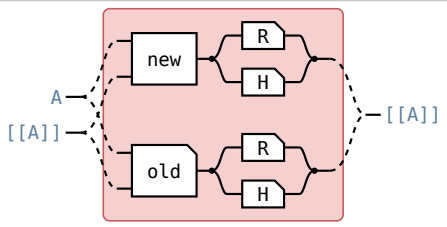
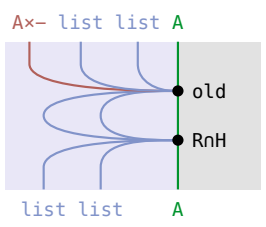
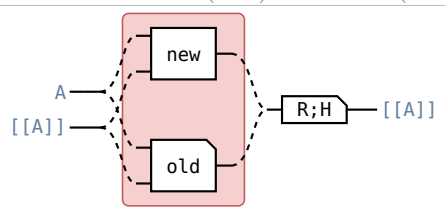
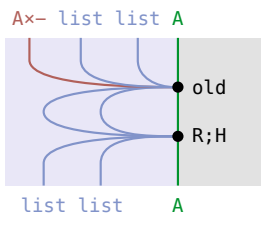


`(𝟙 × (R ∩ H)) old ⊑ old (R ∩ H)`

on a tie the one first segment is a prefix of the other, so prefix-closure of `secure` — Exercise 7.49 `secure prefix ⊑ prefix secure` — lets `old` fire on this side too, and it keeps both the length and the prefix

`(𝟙 × (R;H)) old ⊑ (new ∪ old) (R;H)` (7.17)

gluing the transaction onto a better schedule for the rest gets no further than gluing it on, or calling the van, and bettering the whole schedule after,

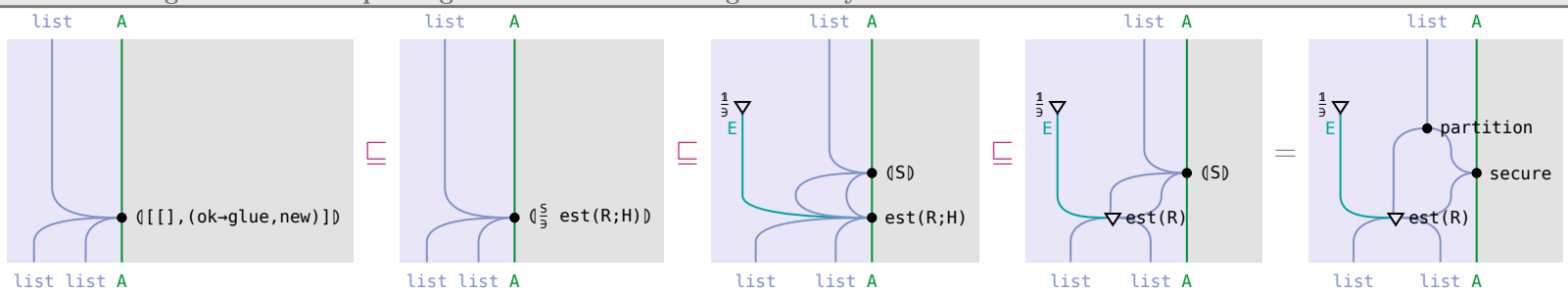
circuit — the `old` branch of each union	**Hinze–Marsden**
	 the `old` operand of `new ∪ old`, in every row
=  `((𝟙 × |R|) old) ∪ ((𝟙 × (R ∩ H)) old)` `R;H=|R| ∪ (R∩H)` — `R;H = |R| ∪ R ∩ H` , `∪` distributes,	
⊑  `(new (R ∩ H)) ∪ (old (R ∩ H))` (7.19) and (7.20) on `|R|` — (7.19) `(𝟙 × ⊤) old ⊑ new H` , (7.20) `(𝟙 × |R|) old ⊑ new R` — and (7.21) on `R∩H` — (7.21) `(𝟙 × (R ∩ H)) old ⊑ old (R ∩ H)`	
⊑  `(new ∪ old) (R;H)` `X∩Y⊑X;Y`, converses	

§7.5.7. Derivation

(∀a. secure[a]) (7.5.7a)

⇒ ⦇[[],(ok→glue,new)]⦈ ⊑ $\frac{\texttt{partition secure´}}{\ni}$ est(R)

the fewest secure segments the transactions can be cut into are one pass along them, the next transaction glued onto the open segment wherever that segment stays secure and the van called where it does not



1	⦇[[],(ok→glue,new)]⦈	[[],(ok→glue,new)] ⊑ $\frac{S}{\ni}$ est(R;H)
2	⊑ ⦇$\frac{S}{\ni}$ est(R;H)⦈	⦇$\frac{S}{\ni}$ est(R;H)⦈ ⊑ $\frac{⦇S⦈}{\ni}$ est(R;H)
3	⊑ $\frac{⦇S⦈}{\ni}$ est(R;H)	R;H ⊑ R
4	⊑ $\frac{⦇S⦈}{\ni}$ est(R)	Exercise 7.51 (∀a. secure[a]) ⇒ partition secure´ = ⦇S⦈
5	= $\frac{\texttt{partition secure´}}{\ni}$ est(R)	